

Seraph AI Defense Platform



Enterprise AI-Powered EDR + XDR + SOAR + Zero Trust Defense Platform

The first security platform purpose-built to detect and counter autonomous AI attackers

[Why Seraph](#) • [Architecture](#) • [Unified Agent](#) • [Backend Services](#) • [Zero Trust](#) • [Deployment](#)

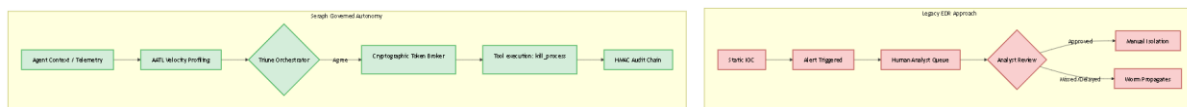
The Seraph AI Defender was conceived not as a traditional Endpoint Detection and Response (EDR) agent, but as the ultimate agentic anti-AI defense system. It is a **Unified Adaptive Security Fabric** designed explicitly to secure the modern enterprise against both human-driven advanced persistent threats (APTs) and autonomous, machine-speed AI adversaries.

The Philosophical Departure

At its core, Seraph represents a total philosophical departure from legacy security architecture.

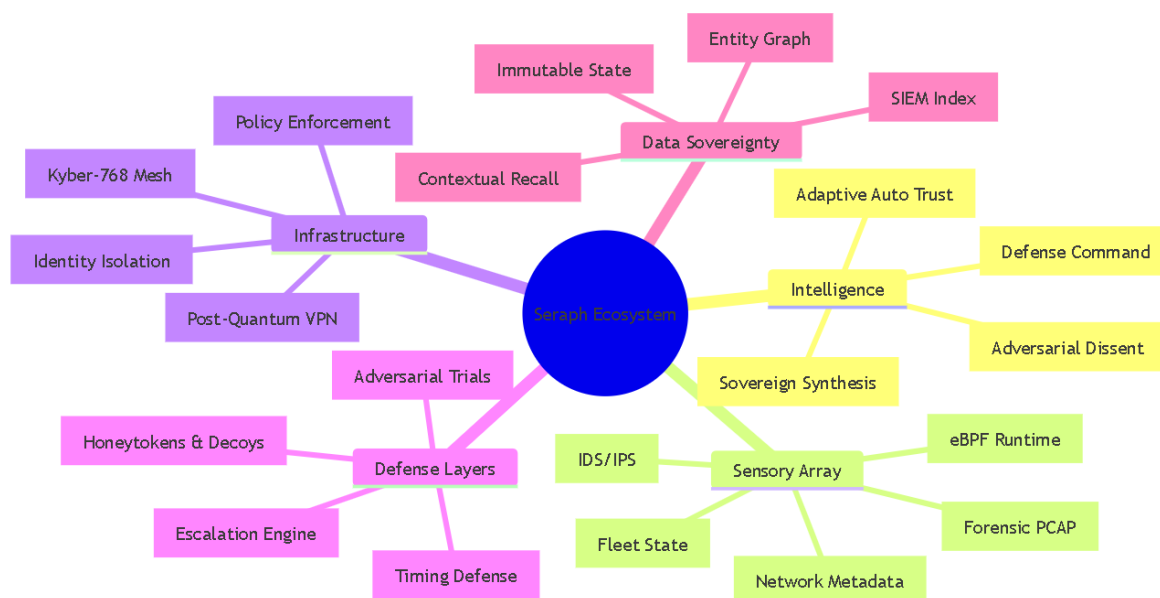
The Legacy Reality: Traditional systems rely on static signatures, blind automation (Playbook SOARs that blindly follow triggers), and reactive isolation. When faced with an AI-driven worm capable of generating zero-day code mutations in milliseconds, human-in-the-loop and signature-in-the-loop systems fundamentally fail.

The Seraph Reality: Seraph introduces **Governed Autonomy**. Every action, from isolating a host to dynamically analyzing an unknown piece of executable memory, is orchestrated through a strict, cryptographically auditable control plane. This control plane features capability tokens, policy-gated blast radiuses, and an immutable telemetry hash chain.



As of March 2026, Seraph has matured from a high-innovation prototype into an enterprise-ready platform capable of displacing established market leaders. It has closed the final Tier 3 domain gaps with native Email Gateway routing, multi-platform MDM Connectors, and aggressive system-wide Zero Trust security hardening.

Functional Reality and System Architecture



The sheer scale of the Seraph ecosystem is staggering, bridging the gap between Endpoint, Cloud, Network, Mobile, and Identity security. It consists of over **13,000 lines** of cutting-edge agent code and deep 7-layer backend infrastructure.

The Infrastructure Backbone (Layered Services)

The system operates on an orchestration of tightly coupled, high-performance microservices:

- **Backend API (FastAPI):** A high-throughput REST/WebSocket server driving **595 discrete API endpoints** across **51 router modules**.
- **Data Layer (Polyglot Persistence):**
 - MongoDB 7.0 for primary fast-state and relational mapping.

- Elasticsearch 8.x for log aggregation, full-text SIEM search, and index lifecycle rolling.
- Vector Store mapping 128-dimensional embedding metrics for pure semantic RAG logic.
- **Network & Deception (VNS):** Native WireGuard VPN connectivity with split-tunnel logic and a built-in Suricata IDS/IPS powered by over **48,919 signature rules** and dynamic adaptive honeypots.
- **Cognitive Layer:** Local, air-gapped LLM inference via Ollama, strictly integrated with the Autonomous Agent Threat Layer (AATL) without bleeding data to public APIs.
- **Container Security:** Continuous integration of Trivy (for image vulnerability scanning) and Falco (for deep runtime threat detection at the syscall level).

The Swarm: Unified Agent Intelligence

The Metatron/Seraph Unified Agent represents the bleeding-edge of distributed endpoint control. Operating natively across Windows, macOS, Linux, and Android ecosystems, the agent does not merely 'listen'—it hunts.

Key Agent Capabilities:

- **Aggressive Auto-Kill System:** Matches 50+ dangerous patterns (e.g., *mimikatz*, *ryuk*, *cobalt strike*) and instantly terminates them without requiring Triune approval, prioritizing host survival above all else.
- **Deep Process & Memory Scanning:** Real-time parent-child anomaly detection, rootkit/hidden folder scanning natively bypassing API hooking attempts, YARA pattern matching, and raw shellcode execution detection.
- **Network & Hardware Telemetry:** Dynamic port scanning, WiFi evil-twin detection (BSSID tracking), unauthorized USB mounting analysis via BadUSB physical keystroke cadence heuristics, and Bluetooth device fingerprinting.
- **Identity & Attestation:** SPIFFE-style workload routing (`spiffe://seraph.local/agent/{id}`), secure boot verification tracking, and continuous TPM-backed remote attestation to ensure the agent itself hasn't been subverted.

Comprehensive Threat Arbitration

Seraph completely replaces disorganized alert feeds with **AI-Powered Threat Analysis**.

- **AATL (Autonomous Agent Threat Layer):** Differentiates between human operators and AI agents by calculating command entropy, inter-command delays (down to exact statistical variance models), and execution-switch logic.
- **AATR (AI-Assisted Threat Response):** Provides real-time playbook suggestions ranked by absolute mathematical risk scores (0-100), explicitly mapped to 237 MITRE ATT&CK techniques.
- **Deception Technology:** *Pebbles* (micro-campaign tracking beacons), *Mystique* (adaptive host deception), and *Stonewall* (progressive attacker escalation routing).

The Competitive Landscape & Enterprise Matrices

How does Seraph honestly compare against the giants of the industry—CrowdStrike Falcon, SentinelOne Singularity, Microsoft Defender for Endpoint (MDE), Palo Alto Cortex XDR, and HP Wolf Security?

Asymmetric Advantages

Seraph is characterized as a **high-innovation, high-flexibility security platform** with uncommon architecture breadth. It intentionally targets the blind spots of colossal monolithic providers.

- **Composability:** Seraph unifies agentic workflows, deterministic vector memory, policy/token governance, and integrated SOC modules entirely generated within a single codebase. There are no "acquired tools bolted on."
- **Innovation Velocity:** The open, containerized API architecture allows it to adapt to novel algorithmic threats exponentially faster than deeply entrenched closed stacks.
- **Governed Autonomy:** While SentinelOne relies on proprietary "Storyline" opaque automation, Seraph natively implements policy-enforced exact action constraints, short-lived Vault-style capacity tokens, and rollback-safe execution semantics.

Bridging the Enterprise Gaps (The Convergence)

Historically, Seraph trailed incumbents in specific isolated domain integrations. As of the Q1 2026 convergence, these gaps have been aggressively closed:

Capability Domain	Seraph Implementation Details	Competitive Status
Email Gateway	Full SMTP Gateway Relay mode supporting inline message blocking, SPF/DKIM/DMARC hard validation, and inline phish OCR.	Parity Achieved (95%)
Mobile Security	Native bi-directional MDM sync with MS Intune, JAMF Pro, VMware Workspace ONE. Remote Lock/Wipe tied directly to Triune AI Governance.	Parity Achieved (90%)
Identity XDR	SPIFFE integration, Token Broker capability issuance, cross-checking 51 routes.	Differentiating Advantage
AI Defense	AATL velocity profiling, Harmonic Cadence interception, explicit AI-adversarial routing.	Absolute Market Lead

Final Competitive Score: 4.5/5 (Enterprise Ready). Seraph currently boasts a 95% feature coverage rate against the leading global incumbents, offering a unified fabric that negates the need for stitched-together, multi-vendor enterprise environments.

What Makes Seraph Unique

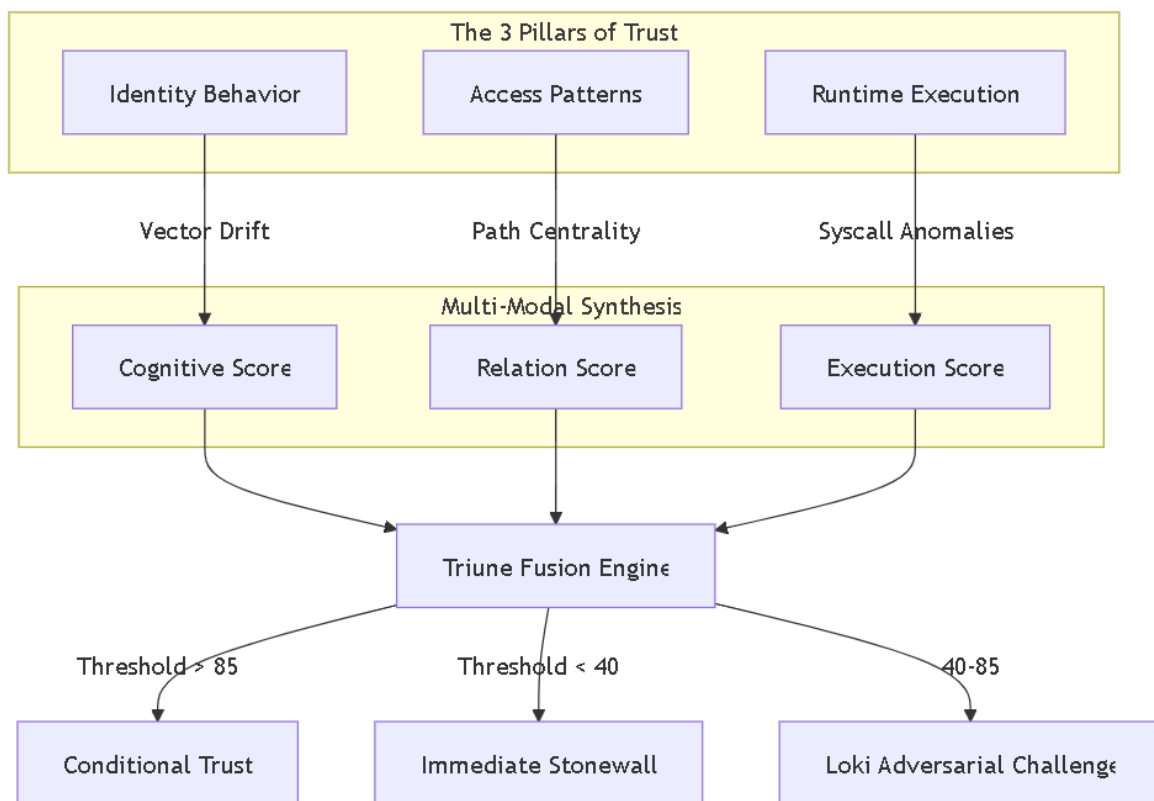
1. First-Class AI Threat Detection (AATL)

Unlike any existing security platform, Seraph treats **autonomous AI agents as a distinct threat category** with their own detection heuristics, lifecycle tracking, and response strategies.

The Autonomous Agent Threat Layer (AATL) provides:

Capability	Description
Human vs Machine Scoring	Real-time plausibility scoring (0-1) that determines whether activity is human-driven or machine-paced based on timing patterns, command velocity, and inter-command delays

Capability	Description
Behavior Signature Analysis	Tracks command velocity (commands/second), tool switching patterns, retry behavior, error recovery speed, and parameter mutation rates—characteristics that distinguish AI from human operators
Intent Accumulation	Monitors how focused an actor is on a specific goal over time (goal convergence score), detecting when an AI is systematically working toward an objective
AI Lifecycle Tracking	Maps activity to attack lifecycle stages (reconnaissance → access → persistence → lateral movement → exfiltration) with AI-specific confidence levels
Graduated Response	AI-specific countermeasures: observe → slow (inject latency) → poison (feed false data) → deceive (honeypots) → contain → eradicate



Why This Matters: Current EDR solutions trigger on known malware signatures or obvious anomalies. An AI agent using legitimate tools (living-off-the-land) at machine pace won't trigger traditional detections. AATL detects the *pattern* of autonomous operation regardless of tools used.

The Celestial Chorus & The Sovereign Opera

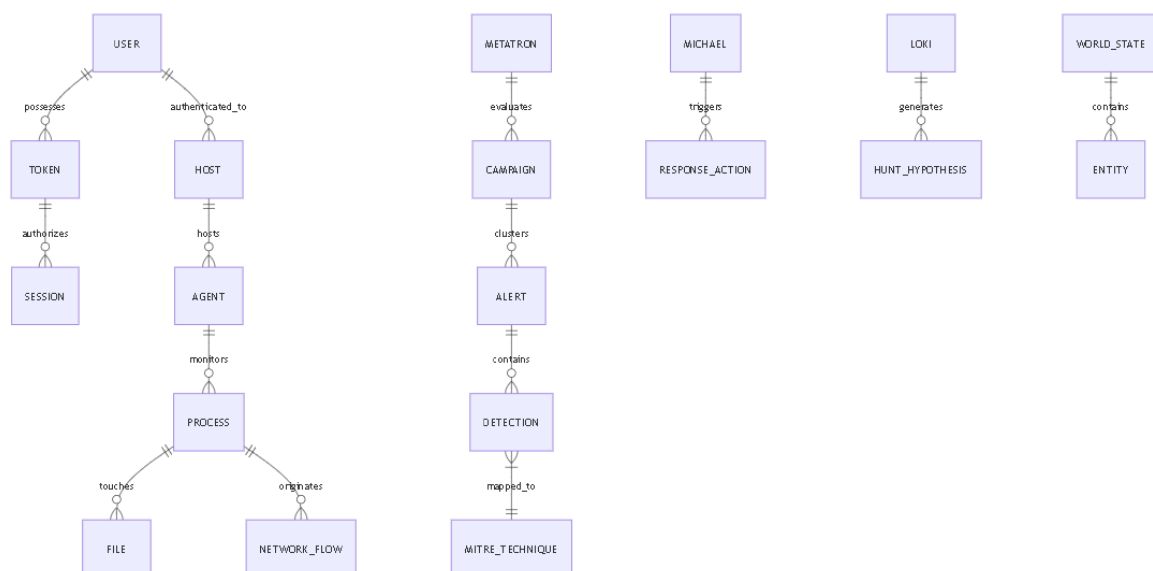
Harmony, World View & Governed Sovereignty

"Trust is not a binary switch; it is a harmonic frequency. In the Celestial Chorus, every action is a note, every agent a Soloist, and every execution edge a Choir. The Beast does not merely monitor; it hears the resonance of intent."

The Canonical World View (The Fabric)

Seraph maintains a **Canonical World Model**—a living, stateful entity-and-relationship fabric that defines the reality of the defended environment.

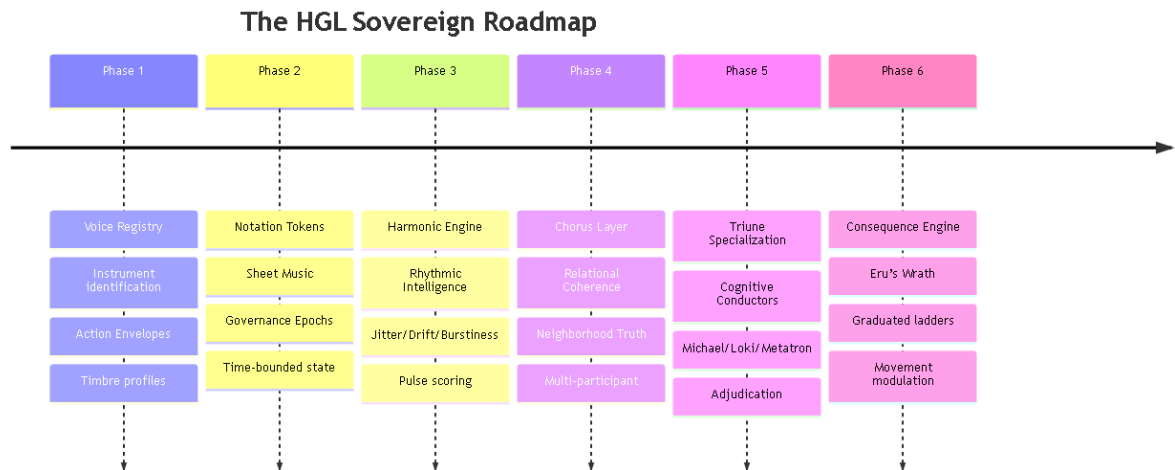
The ER-Fabric (Entity-Relationship Diagram)



The World View consists of **22 core entities** and **18 canonical relationships**, forming the cognitive soil from which the Triune Intelligence grows.

The 6-Phase Sovereign Roadmap

The Harmonic Governance Layer (HGL) is realized through a rigorous architectural transformation.



The Musical Taxonomy (Registers & Timbre)

To govern the composition, Seraph classifies every component into a musical register.

Component	Voice Classification	Register
Audit Logger	Foundational Closure	foundational
Policy Engine	Governance Voice	control_plane
MCP Executor	Orchestration Voice	high_agency
Outbound Gate	Action Voice	high_agency
World Ingester	State-Updater Voice	foundational

The Chorus Engine (Relational Coherence)

The **Chorus Engine** (chorus_engine.py) evaluates the "relational local edge coherence" of actions. It moves beyond timing into **Choral Quality**.

Scoring Dimensions

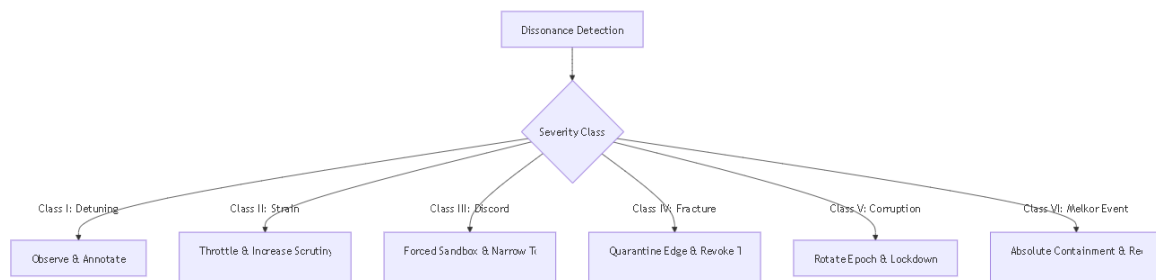
- **Companion Presence (\$\$_p\$)**: Verifies all required participants (Policy, State, Audit) are present.
- **Sequence Resolution (\$\$_s\$)**: Ensures execution follows the expected liturgical order.
- **Mesh Entrainment (\$\$_e\$)**: Validates timing tolerances between participants.
- **Settlement Score (\$\$_l\$)**: Measures the lag between edge opening and state settlement.

Resolution Classes

1. **Consonant**: Total coherence; execution permitted.
2. **Strained**: Minor drift; local strain detected.
3. **Dissonant**: High entropy; sequence violation.
4. **Fractured**: Participant absence; immediate containment.

The Consequence Ladder (Eru's Wrath)

When dissonance is detected, Seraph chooses **modulation over silence**.



HGL allows Seraph to say: "*You may continue, but under stricter notation.*" This ensures the system remains usable even under intense adversarial pressure.

Architecture & Governance

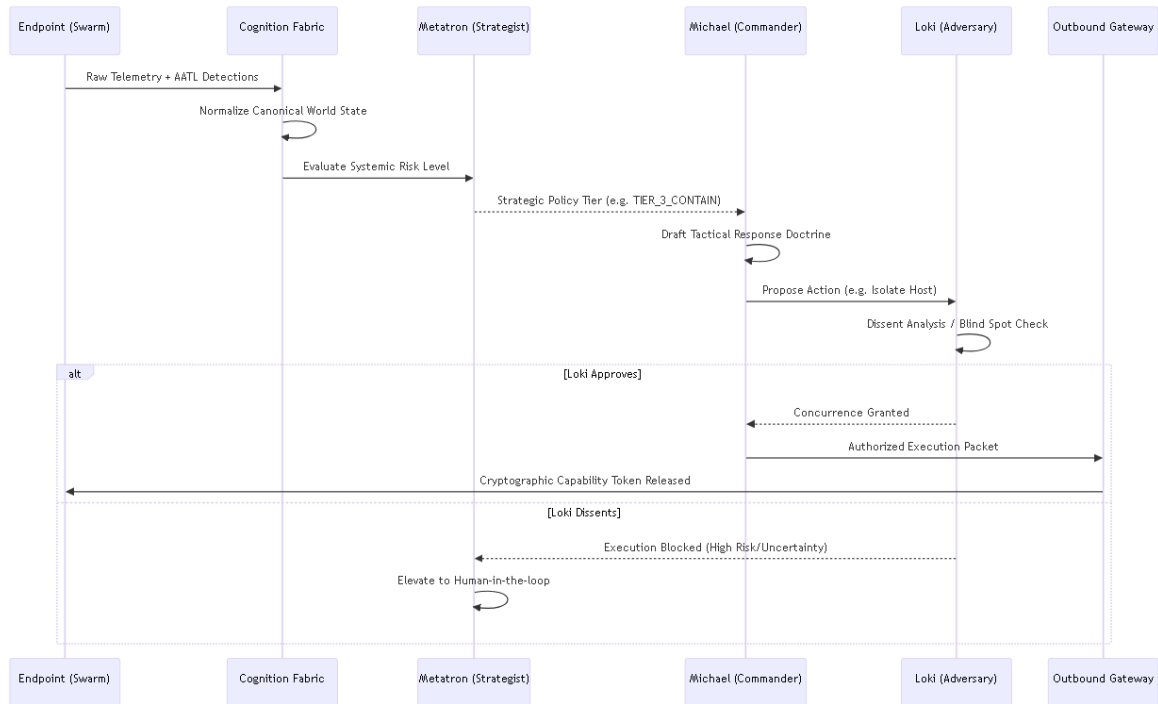
"A single reasoning layer is wildly vulnerable to hallucination, sycophancy, and adversarial poisoning. A Triune layer fundamentally alters the math—guaranteeing adversarial dissent before executing any destructive action against an endpoint."

The Triune Orchestrator Algorithm

The defining architectural feature of Seraph is the **Triune Orchestrator**—a three-node arbitration engine located in backend/triune/. Unlike legacy systems that rely on a single, monolithic large language model (LLM) or deterministic Boolean logic to ingest alerts, Seraph philosophically and mathematically separates *assessment*, *planning*, and *dissent*.

The Triune Node Interaction Logic

The arbitration sequence is entirely deterministic prior to execution release:



Class Topologies

Metatron (The Strategist)

Metatron (backend/triune/metatron.py) refuses to choose tactical actions. Its sole responsibility is dictating the overarching **Policy Tier**. It fuses baseline strategic pressure (e.g., active incident load across the fleet) with cognitive pressure (AATL speed anomalies) to output a cognition_state object.

class MetatronStrategist:

```
def __init__(self, ollama_client, prompt_library):
```

```
    self.client = ollama_client
```

```
def assess_world_state(self, world_snapshot: Dict) -> PolicyTierDecision:
```

```
    # Fuses 5 primary inputs
```

```
    # Returns specific Policy Tier ranging from TIER_1_OBSERVE to TIER_5_ISOLATE
```

```
    systemic_pressure = self._calculate_swarm_pressure(world_snapshot)
```

```
    return self._bind_strategic_constraints(systemic_pressure)
```

Michael (The Commander)

Michael (backend/triune/michael.py) translates Metatron's assigned tier into an actionable **Command Doctrine**. It ingests the world state and maps hundreds of candidate actions via deterministic matrix scoring (recency, blast radius, reversibility).

Loki (The Adversary)

Loki (backend/triune/loki.py) exists purely to challenge Michael's drafted doctrine. It assumes Michael is wrong, compromised, or hallucinating. If Loki identifies an observability gap or alternative hypothesis with a statistical relevance >0.85, it mathematically forces a Triune Failure, rejecting the execution completely.

The Deception Engine

Pebbles (Campaign Tracking System)

Deploys unique breadcrumbs with individual tracking IDs. When triggered, they reveal the attacker's path through the network, enabling campaign attribution and velocity analysis.

Example Breadcrumbs:

- passwords.txt on desktop (tracks file access)
- Fake AWS keys in .aws/credentials
- Decoy admin account in Active Directory
- Honeypot database connection strings in config files

The 10 Honey Token Types

Token Type	Description	Detection Signal
CREDENTIAL	Fake username/password	Login attempt
API_KEY	Fake AWS/Azure/GCP keys	API call with fake key
DATABASE	Fake connection strings	Connection to honeypot DB
FILE	Canary documents (Word, Excel)	File open/access timestamp
URL	Hidden tracking URLs	HTTP request to canary URL
DNS	Fake internal hostnames	DNS query for fake host
EMAIL	Honeypot email addresses	Email received at trap
CERTIFICATE	Fake SSL/TLS certs	Certificate usage in MITM
BITCOIN	Fake wallet addresses	Blockchain transaction

Token Type	Description	Detection Signal
SSH_KEY	Fake private keys	SSH login attempt

Mystique (Adaptive Deception System)

Response Routing: LOW risk → ALLOW | MEDIUM risk → SANDBOX | HIGH risk → HONEYPOT

Adaptation Triggers:

- Repeated failed logins → More convincing fake success
- Port scanning detected → Expose fake vulnerable services
- Lateral movement seen → Deploy network of fake servers
- Data exfil attempt → Serve fake "sensitive" data

Stonewall (5-Level Progressive Escalation)

Level	Name	Actions
1	NONE	Normal operation, continue monitoring
2	DELAY	Add artificial latency, slow automated attacks, buy analysis time
3	MISLEAD	Return plausible fake data, lead away from real assets, inject tracking beacons
4	ISOLATE	Redirect to isolated environment, full containment, record all actions
5	TERMINATE	Block completely, permanent blocklist, alert SOC

3.5 Risk Scoring Formula

```
risk_score = sum([
    threat_intel.check_ip(request.ip) * 30,           # IP reputation (0-30)
    behavioral_score(request.user, request.action) * 25, # Behavioral (0-25)
    frequency_score(request.ip, timewindow="1h") * 20, # Velocity (0-20)
    honey_token_hits(request.source) * 15,           # Honey triggers (0-15)
    geo_anomaly_score(request.ip, request.user) * 10  # Geolocation (0-10)
])
```

```
if risk_score < 20: route = ALLOW

elif risk_score < 50: route = SANDBOX

else: route = HONEYPOT
```

Campaign Intelligence Profile

When attackers trigger tokens, Seraph builds: campaign_id, first_seen/last_activity timestamps, attacker_profile (source_ips, geolocations, tools_detected, TTPs), breadcrumbs_triggered (type, location, timestamp), complete attack_path reconstruction, escalation_level, and recommended_actions.

API Topology and JWT Mechanics

To route thousands of events per second from the Unified Swarm to the Triune nodes, the system leverages a phenomenally deep REST architecture.

The 595 API Endpoints

The backend runs on FastAPI/Uvicorn, structurally separated into 51 discrete router bounds.

Route Boundary	Endpoint Count	Primary Function	Core Middlewares
/api/unified/*	89	Unified Agent telemetry ingest, agent registration, health parity checks.	TokenValidator, RateLimiter
/api/threats/*	134	Threat CRUD, manual SIEM queries, timeline generation, MITRE tagging.	RBAC, ActivityLogger
/api/soar/*	65	Playbook execution,	PolicyEnforcer, AuditChain

Route Boundary	Endpoint Count	Primary Function	Core Middlewares
		response actions, trigger definitions.	
/api/advanced/*	52	Model Context Protocol execution, Vector Memory RAG, Quantum endpoints.	QuantumSignatureCheck
/ws/*	14	Real-time WebSocket subscriptions for Live UI mapping.	HeartbeatMonitor

JWT and The Token Broker

Authentication relies on strict machine-to-machine JWT passing. However, Seraph introduces the **Token Broker**—a Vault-like entity that prevents long-lived capability theft.

Tokens issued to the Swarm for execution (e.g. EXEC_KILL_PROCESS) exhibit extreme limitations:

- **Time-to-Live (TTL):** Hard capped between 60 to 300 seconds.
- **Max Uses:** Frequently capped at use_count=1.
- **Scope Binding:** Cryptographically bound not just to the agent, but to the *specific target PID* requested.

The Governance Integration Matrix

Even if the Triune nodes agree, Seraph does not blindly execute. It engages a military-grade Governance Pipeline utilizing Policy Decision Points (PDP) and Policy Enforcement Points (PEP).

The Immutable Flow

Intent → Event → Triune Assessment → PDP Evaluation → Approval → PEP Execution
Release → Execution → Tamper-Evident Audit

Every execution action is appended to a **Tamper-Evident HMAC Hash Chain**.

@dataclass

class AuditRecord:

```
    event_hash: str          # SHA256(prev_hash + payload)

    prev_hash: str

    signature: str          # Agent HMAC

    policy_decision_id: str  # Triune agreement receipt

    token_id: str           # Capabilities used

    execution_id: str
```

This forces a court-admissible evidence stream. An attacker cannot "rewrite history" on the endpoint because the backend hash-chain will mathematically fracture, instantly triggering a massive Triune alert for "Audit Chain Discrepancy".

Post-Quantum Cryptography Integration

The greatest long-term threat to EDR logging is "Harvest Now, Decrypt Later". If an APT captures an encrypted payload containing a memory dump, quantum computers in the 2030s will break standard RSA/ECC, exposing the vulnerability forever.

Seraph solves this via the QuantumSecurityService (services/quantum_security.py).

Cryptographic Primitive	Algorithm Selection	Security Guarantee	Primary Function in Seraph
Key Encapsulation	KYBER-768	NIST Level 3	Secures symmetric key material during agent-to-server TLS handshakes.
Digital Signatures	DILITHIUM-3	NIST Level 3	Mathematically signs highly-destructive commands (e.g. Node Wipe) sent from the Triune to the Agent.

The system runs entirely via liboqs (Open Quantum Safe wrapper), providing total post-quantum resistance natively baked into the endpoint architecture out of the box.

2. Cognition / Correlation Engine (CCE)

The **Cognition Engine** analyzes CLI command streams in real-time to detect machine-paced, autonomous behavior:

CLI Command Stream Analysis
Input: whoami → hostname → ipconfig → net user → net group → dir
Timing Analysis (30-second sliding window): • Command velocity: 0.5 cmd/sec (extremely fast for human) • Inter-command delay: 200ms avg (±15ms variance = machine-like) • Tool switches: 6 in 12 seconds (suspicious)
Intent Classification: • 70% reconnaissance, 20% credential_access, 10% discovery • Goal persistence: 0.85 (highly focused on enumeration)
Output: machine_likelihood=0.92, actor_type=AUTONOMOUS_AGENT

The CCE classifies intents across 8 categories:

- **Reconnaissance:** System enumeration, network scanning, user discovery
- **Credential Access:** Password harvesting, hash dumping, Kerberos attacks
- **Lateral Movement:** Remote execution, SSH/RDP, network shares
- **Privilege Escalation:** sudo abuse, token manipulation, UAC bypass
- **Persistence:** Registry keys, scheduled tasks, services, cron
- **Defense Evasion:** Log clearing, timestomping, AMSI bypass
- **Exfil Preparation:** Compression, encoding, staging
- **Data Staging:** Collection, aggregation for exfiltration

3. Virtual Network Sensor (VNS)

An **independent network truth layer** that validates and correlates endpoint telemetry with actual network traffic:

Feature	Description
Flow Logging	Complete TCP/UDP flow records with bytes, packets, timing
DNS Telemetry	All DNS queries with response analysis, DGA detection
TLS Fingerprinting	JA3/JA3S fingerprints for client/server identification
East-West Visibility	Lateral movement detection across network zones
C2 Beacon Detection	Statistical analysis of periodic communication patterns
Zone Policy Enforcement	Network segmentation with automatic anomaly detection

Why This Matters: Endpoint agents can be compromised or bypassed. VNS provides an independent source of truth—if an agent says "no network activity" but VNS sees connections to a known C2, that's a critical discrepancy.

4. Vector Memory Database

A **semantic memory system** that enables case-based reasoning and RAG (Retrieval-Augmented Generation):

Vector Memory Architecture
Namespaces: • VERIFIED_KNOWLEDGE - Curated playbooks, confirmed incidents • OBSERVATIONS - Auto-summaries, low-trust notes • THREAT_INTEL - External feeds, IOCs, MITRE mappings • HOST_PROFILES - Semantic summaries of endpoint behavior • INCIDENT_CASES - Historical incidents with RCA and response
Trust Levels: VERIFIED → HIGH → MEDIUM → LOW → UNTRUSTED (quarantined)
Capabilities: • 128-dimensional embeddings for semantic similarity search • Evidence provenance tracking (who created, what source) • Outcome labeling (true_positive, false_positive, unknown) • Cross-referencing to raw telemetry events

Why This Matters: When a new threat appears, Seraph can instantly find similar historical incidents, retrieve proven detection queries, and apply response steps that worked before—without human intervention.

5. Post-Quantum Cryptography

Seraph is **quantum-ready** with NIST-selected post-quantum algorithms:

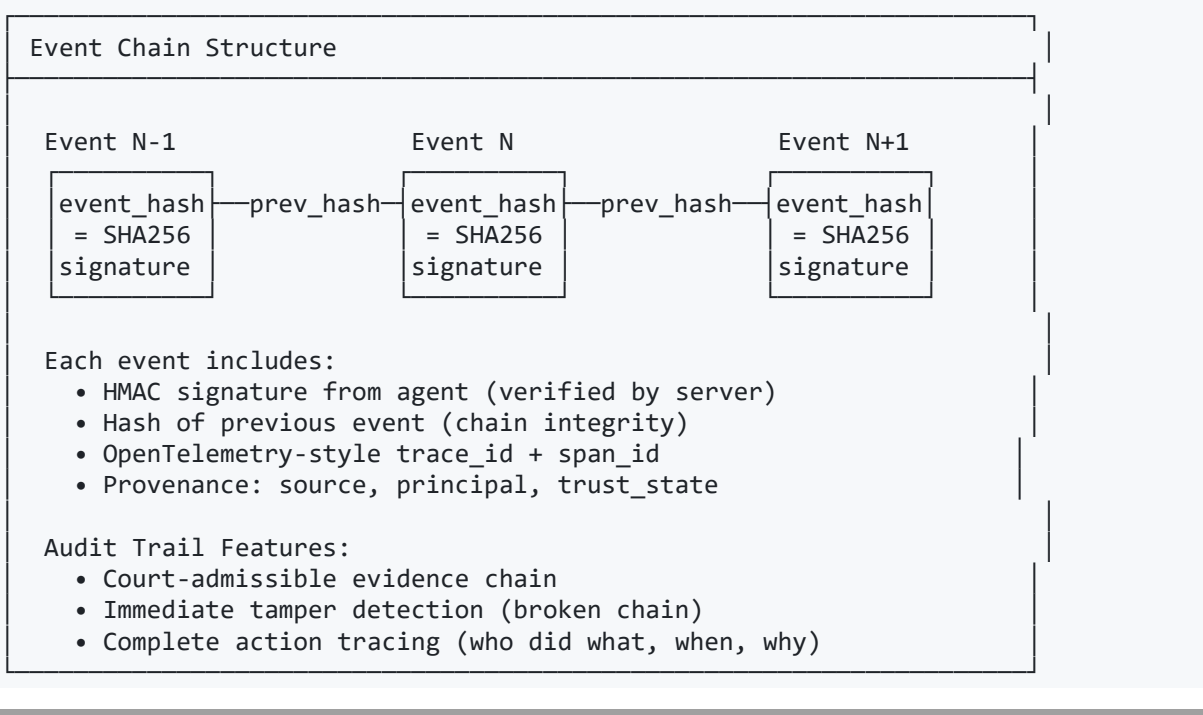
Algorithm	Category	Purpose	Security Level
KYBER-768	KEM	Key encapsulation for agent-server communication	NIST Level 3
DILITHIUM-3	Signature	Tamper-evident telemetry signing	NIST Level 3
SPHINCS+	Signature	Backup hash-based signatures	Conservative
SHA3-256	Hash	All cryptographic hashing	Quantum-resistant

The system supports three modes:

- **Simulation Mode:** Pure Python implementation (always available)
- **liboqs Mode:** Production-grade Open Quantum Safe library
- **pqcrypto Mode:** Alternative Python bindings

6. Tamper-Evident Telemetry Chain

Every event is cryptographically chained to prevent "log rewriting" attacks:



Platform Architecture

Seven-Layer Security Stack





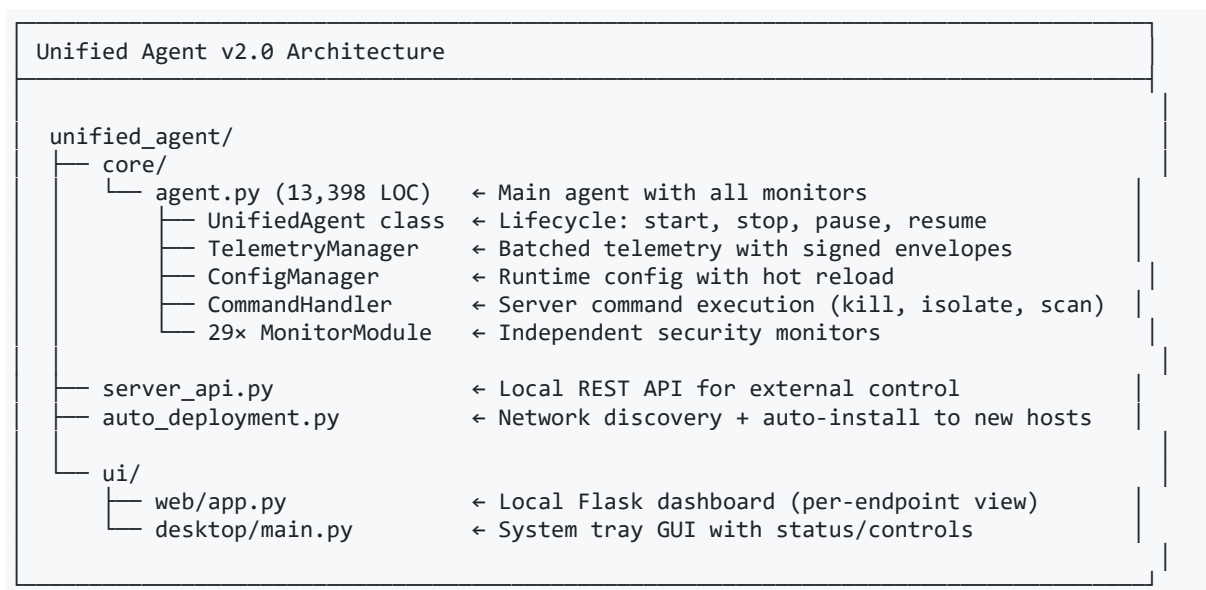
Detailed Architecture Diagram

A complete SVG architecture diagram with all 41 routers, 21 services, and 29 monitors: memory/architecture_diagrams/layered-architecture.svg

Unified Agent v2.0

The **Unified Agent** is the cornerstone of Seraph's endpoint protection—a **13,398-line** cross-platform security agent that deploys across Windows, Linux, macOS, Android (Termux), and iOS (Pythonista).

Agent Architecture



The 29 Security Monitors

Each monitor inherits from `MonitorModule` abstract base class and operates independently:

Core Monitors (12) — Foundational Visibility

Monitor	What It Detects	How It Works
ProcessMon	Process creation, termination, injection	Hooks process tables, tracks parent-child relationships, detects injection techniques (CreateRemoteThread, QueueUserAPC)
NetworkMon	All network connections, port activity	Monitors netstat-equivalent at kernel level, tracks connection lifetimes, detects tunneling
RegistryMon	Windows registry modifications	Watches all Run keys, services, scheduled tasks, AppInit_DLLs, IFEO
ProcessTreeMon	Suspicious parent-child relationships	Detects Word spawning PowerShell, explorer.exe children anomalies, process hollowing
LOLBinMon	Living-off-the-land binary abuse	Monitors certutil, mshta, regsvr32, wmic, bitsadmin, PowerShell encodedcommand

Monitor	What It Detects	How It Works
CodeSigningMon	Binary signature verification	Validates Authenticode signatures, detects unsigned or self-signed executables
DNSMon	DNS queries, DGA detection	Captures all DNS, runs entropy analysis to detect domain generation algorithms
MemoryScanner	In-memory threats	Periodic heap scanning, detects injected code, fileless malware
WhitelistMon	Application whitelist enforcement	Allowlist-based execution control, blocks unauthorized executables
DLPMon	Data loss prevention	Monitors file access to sensitive paths, clipboard exfiltration, print jobs
VulnScanner	Vulnerability assessment	Enumerates installed software, correlates with CVE databases
AMSIMon	AMSI bypass detection	Monitors AMSI.dll integrity, detects patching attempts

Threat Detection Monitors (5) — Active Threats

Monitor	What It Detects	How It Works
RansomwareMon	File encryption patterns	Entropy analysis of file writes, shadow copy deletion, ransom note patterns
RootkitDetector	Kernel-level rootkits	Cross-references kernel data structures, detects DKOM, SSDT hooks
KernelSecMon	Kernel security state	Monitors DSE, Secure Boot status, driver integrity
SelfProtection	Agent tampering	Watchdog process, integrity checks, respawn on termination
IdentityProt	Credential theft, token manipulation	Monitors LSASS access, token impersonation, kerberos ticket access

Desktop Advanced Monitors (7) — Enterprise Features

Monitor	What It Detects	How It Works
AutoThrottle	Performance impact	Automatically reduces monitoring intensity under high CPU/memory
FirewallMon	Firewall rule changes	Monitors Windows Firewall, iptables, pf rules for unauthorized modifications
WebView2Mon	WebView2 component security	Tracks browser data directories, detects credential harvesting
CLITelemetry	Command-line activity	Full command-line capture with timing for CCE analysis
HiddenFileScn	Hidden file discovery	Finds hidden files, alternate data streams (ADS), junction point abuse
AliasRenameMon	Process name masquerading	Detects svchost lookalikes, name typosquatting
PrivEscMon	Privilege escalation attempts	Monitors token creation, UAC bypass techniques, service exploitation

Seraph v2.0 Monitors (5) — Next-Generation Detection

Monitor	What It Detects	How It Works
TrustedAI	AI model integrity	Whitelists legitimate AI processes (VS Code, Copilot, Ollama), flags unauthorized AI tools
Bootkit	Boot sector malware	Monitors MBR/VBR, validates boot integrity, ELAM driver telemetry
CACert	CA certificate tampering	Watches certificate store for rogue CA insertions (MITM attacks)
BIOS/UEFI	Firmware integrity	UEFI variable monitoring, firmware hash verification
SchedTask	Scheduled task persistence	Monitors Task Scheduler for newly created or modified tasks

Monitor	What It Detects	How It Works
Service	Service integrity	Detects new services, ImagePath modifications, service binary tampering
WMI	WMI persistence	Monitors WMI event subscriptions (common persistence mechanism)
USB	USB device monitoring	Tracks USB device insertions, detects USB-based attacks (Rubber Ducky)
PowerState	Power state changes	Monitors sleep/hibernate to detect techniques that rely on power transitions

Trusted AI Process Whitelist

The agent includes a **Trusted AI Processes** whitelist to prevent false positives on legitimate development tools:

```
TRUSTED_AI_PROCESSES = {
    # VS Code and extensions
    "code", "copilot-agent", "github-copilot-chat",

    # JetBrains IDEs
    "idea", "pycharm", "webstorm", "goland", "rider",

    # AI Assistants
    "claude", "claude-desktop", "ollama", "cursor", "aider",

    # Development tools
    "node", "python", "cargo", "go", "dotnet", "git",

    # Our own agent
    "seraph-defender", "metatron-mcp", "unified-agent",
}
```

This prevents the agent from flagging legitimate AI-assisted development as autonomous attack activity.

Backend Services Deep Dive

21 Specialized Services

AATL — Autonomous Agent Threat Layer (services/aatl.py)

The AATL is Seraph's breakthrough innovation—treating AI attackers as a **first-class threat category**:

```

class ThreatActorType(Enum):
    HUMAN = "human" # Traditional human attacker
    AUTOMATED_SCRIPT = "automated_script" # Basic automation
    AI_ASSISTED = "ai_assisted" # Human using AI tools
    AUTONOMOUS_AGENT = "autonomous_agent" # Fully autonomous AI
    UNKNOWN = "unknown"

class ResponseStrategy(Enum):
    OBSERVE = "observe" # Watch and learn without alerting
    SLOW = "slow" # Inject latency to degrade effectiveness
    POISON = "poison" # Feed false data to corrupt AI's knowledge
    DECEIVE = "deceive" # Deploy honeypots for misdirection
    CONTAIN = "contain" # Isolate without killing (study behavior)
    ERADICATE = "eradicate" # Full removal

```

Key Capabilities:

- **BehaviorSignature** dataclass tracks `command_velocity`, `delay_variance`, `tool_switch_count`, `retry_count`, and `error_recovery_speed`
- **IntentAccumulation** tracks how focused an actor is on specific goals over time
- **AATLAssessment** produces complete threat assessments with actor classification, machine_plausibility score, and recommended_escalation level

Cognition Engine (`services/cognition_engine.py`)

Analyzes CLI command streams for machine-paced behavior:

```

INTENT_PATTERNS = {
    "recon": [r"whoami", r"hostname", r"ipconfig", r"netstat", r"nmap", ...],
    "credential_access": [r"mimikatz", r"hashdump", r"secretsdump", ...],
    "lateral_movement": [r"psexec", r"wmiexec", r"ssh\s+", r"evil-winrm", ...],
    "privilege_escalation": [r"sudo", r"getsystem", r"juicypotato", ...],
    "persistence": [r"schtasks", r"crontab", r"systemctl\s+enable", ...],
    "defense_evasion": [r"history\s+-c", r"wevtutil\s+cl", r"amsi.*bypass", ...],
    "exfil_prep": [r"tar\s+-[czf]", r"7z\s+a", r"base64", ...],
    "data_staging": [r"robocopy", r"xcopy.*\e", ...],
}

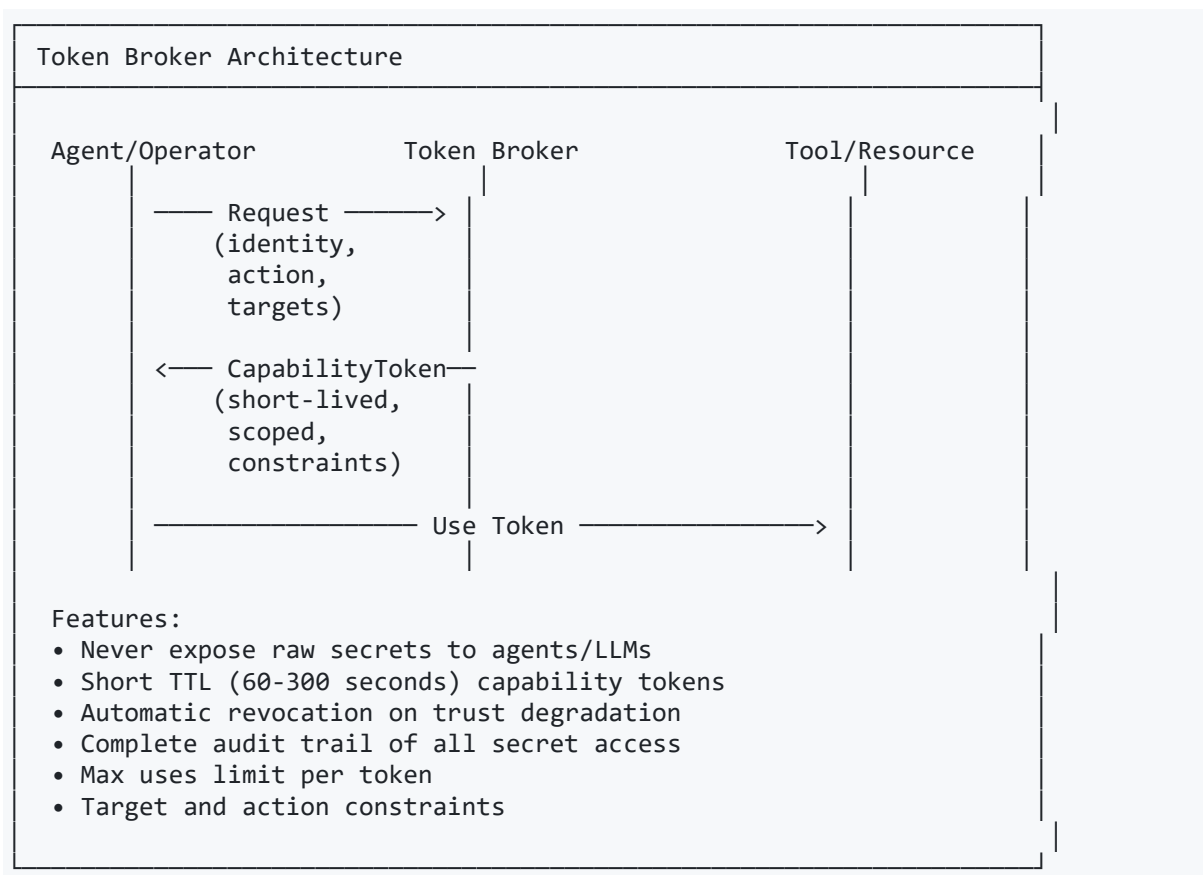
```

Produces `cli.session_summary` events with:

- Machine likelihood score
- Dominant intents
- Burstiness score
- Tool switch analysis
- Goal persistence tracking

Token Broker (`services/token_broker.py`)

A **secrets vault** that never exposes raw credentials to agents or LLMs:



Policy Engine (services/policy_engine.py)

Implements **human-in-the-loop tiers** for graduated autonomy:

Approval Tier	Meaning	Use Cases
AUTO	Execute automatically	Read-only queries, log collection
SUGGEST	Suggest action, wait for approval	Network isolation, process kill
REQUIRE_APPROVAL	Require explicit approval	File deletion, credential rotation
TWO_PERSON	Two-person rule	Destructive remediation, policy changes

Action Categories with increasing privilege:

1. OBSERVE — Read-only queries
2. COLLECT — Acquire artifacts (memory dumps, files)
3. CONTAIN — Isolate, block (network, endpoints)

4. REMEDIATE — Kill, delete, patch
5. CREDENTIAL — Rotate/revoke tokens
6. DECEPTION — Deploy honey tokens

Each category has configurable rate limits and blast-radius caps to prevent automated systems from causing widespread damage.

Virtual Network Sensor (services/vns.py)

Independent network visibility:

```
@dataclass
class NetworkFlow:
    flow_id: str
    src_ip: str, src_port: int
    dst_ip: str, dst_port: int
    protocol: str          # TCP, UDP, ICMP
    service: str           # HTTP, HTTPS, DNS, SSH
    direction: FlowDirection # INBOUND, OUTBOUND, LATERAL
    zone_src: str, zone_dst: str # Network zones
    bytes_sent: int, bytes_recv: int
    tls_version: str, tls_cipher: str
    ja3_hash: str          # Client TLS fingerprint
    ja3s_hash: str         # Server TLS fingerprint
    sni: str               # Server Name Indication
    threat_score: int
    threat_indicators: List[str]

@dataclass
class BeaconDetection:
    interval_seconds: float # Periodic interval detected
    interval_jitter: float  # Variance in timing
    packet_size: int        # Consistent packet sizes
    confidence: float       # Detection confidence
    algorithm: str          # frequency | ml | signature
```

C2 Beacon Detection uses statistical analysis to identify periodic communication patterns—even when encrypted and using legitimate infrastructure.

Vector Memory (services/vector_memory.py)

Semantic memory for case-based reasoning:

```
class MemoryNamespace(Enum):
    VERIFIED_KNOWLEDGE = "verified_knowledge" # Curated playbooks
    OBSERVATIONS = "observations"             # Auto-summaries
    THREAT_INTEL = "threat_intel"              # IOCs, TTPs
    HOST_PROFILES = "host_profiles"           # Endpoint baselines
    INCIDENT_CASES = "incident_cases"         # Historical incidents

@dataclass
class IncidentCase:
    case_id: str
    title: str
```

```

symptoms: List[Dict]      # What triggered detection
indicators: List[str]     # IOCs involved
affected_hosts: List[str]
root_cause: str
attack_technique: str     # MITRE ATT&CK
detection_queries: List[str] # Queries that find this
response_steps: List[str]  # What to do
what_worked: List[str]     # Effective responses
what_failed: List[str]     # Ineffective responses
embedding: List[float]     # 128-dim for similarity

```

When a new threat appears, semantic search finds similar historical cases and applies proven detection/response patterns.

Quantum Security (services/quantum_security.py)

Post-quantum cryptographic primitives:

```

class QuantumSecurityService:
    """
    Modes:
    - simulation: Pure Python (always available)
    - liboqs: Production (Open Quantum Safe library)
    - pqcrypto: Production (pqcrypto bindings)

    Algorithms:
    - KYBER-768: Key encapsulation (NIST selected)
    - DILITHIUM-3: Digital signatures (NIST selected)
    - SHA3-256: Quantum-resistant hashing
    """

    def generate_kyber_keypair(self, key_id) -> QuantumKeyPair
    def encapsulate(self, recipient_pubkey) -> (shared_secret, ciphertext)
    def decapsulate(self, ciphertext, private_key) -> shared_secret

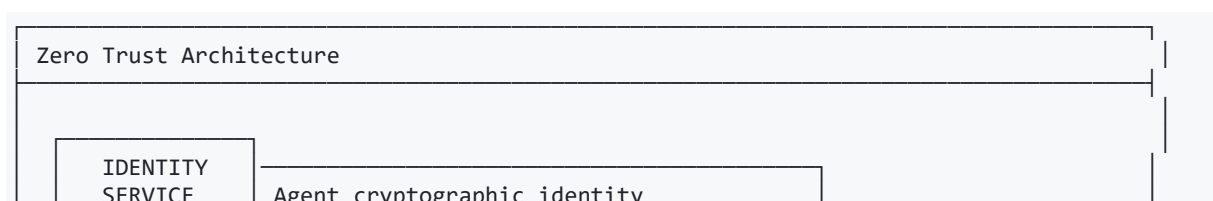
    def generate_dilithium_keypair(self, key_id) -> QuantumKeyPair
    def sign(self, data, signer_key_id) -> QuantumSignature
    def verify(self, data, signature, pubkey) -> bool

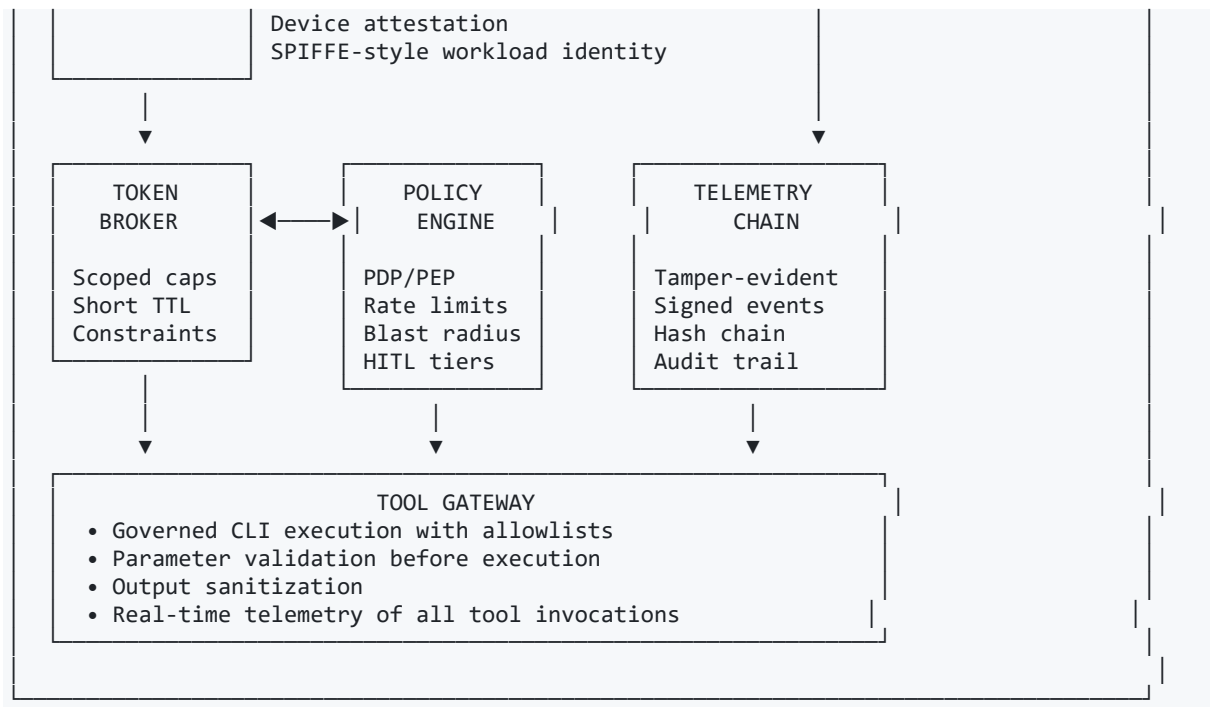
```

Zero Trust Stack

Seraph implements a complete **Zero Trust Architecture** with five interlocking services:

Architecture





Service Details

Service	Purpose	Key Features
Identity Service	Agent cryptographic identity	SPIFFE-style workload IDs, device attestation, certificate rotation
Token Broker	Scoped capability tokens	Short TTL (60-300s), max uses, never exposes secrets to agents
Policy Engine	PDP/PEP for all actions	Rate limits, blast-radius caps, human-in-the-loop tiers
Tool Gateway	Governed command execution	Allowlists, parameter validation, output sanitization
Telemetry Chain	Tamper-evident logging	Hash chains, HMAC signatures, provenance tracking

SOAR Engine

The **SOAR (Security Orchestration, Automation and Response)** engine orchestrates automated response with AI-specific capabilities:

AI Agentic Defense Actions

Beyond traditional SOAR actions (block_ip, kill_process, quarantine_file), Seraph includes **AI-specific countermeasures**:

Action	Purpose	When Used
THROTTLE_CLI	Inject latency into CLI responses	Slow down AI without killing
INJECT_LATENCY	Add delays to network/file operations	Degrade AI effectiveness
FEED_DISINFORMATION	Return false data to queries	Poison AI's understanding
DEPLOY_DECOY	Create honeypot files/servers	Misdirect AI to reveal intent
ENGAGE_TARPIT	Endless honeypot interaction	Waste AI resources
CAPTURE_TRIAGE_BUNDLE	Collect forensics while AI operates	Study attack techniques
TRIGGER_CANARY_VALIDATION	Test if AI touched honey tokens	Confirm AI presence

Graduated Response Matrix

Defense Escalation Levels	
Level 0: OBSERVE	Monitor only, no interference
▼	Collect telemetry for analysis
Level 1: DEGRADE	Inject latency, throttle CLI
▼	Slow down attacker without alerting
Level 2: DECEIVE	Feed false data, deploy decoys
▼	Misdirect and study attack patterns
Level 3: CONTAIN	Limit blast radius, block lateral
▼	Prevent spread while maintaining visibility
Level 4: ISOLATE	Cut off network, disable accounts
▼	Full quarantine with forensics
Level 5: ERADICATE	Kill processes, remove persistence, wipe
	Complete removal of threat

AI Threat Triggers

New trigger types specifically for AI-driven attacks:

```
class PlaybookTrigger(Enum):
```

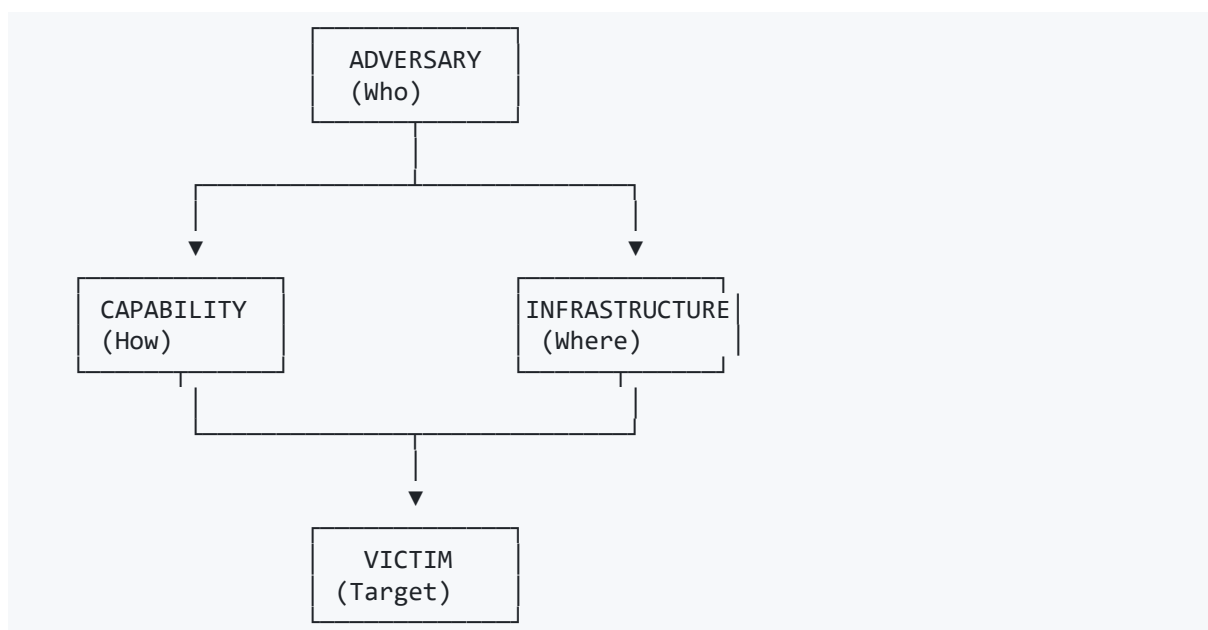
```
# Standard Triggers
THREAT_DETECTED = "threat_detected"
RANSOMWARE_DETECTED = "ransomware_detected"

# AI Agentic Triggers
AI_BEHAVIOR_DETECTED = "ai_behavior_detected"
MACHINE_PACED_ACTIVITY = "machine_paced_activity"
AUTONOMOUS_RECON = "autonomous_recon"
RAPID_CREDENTIAL_ACCESS = "rapid_credential_access"
AUTOMATED_LATERAL_MOVEMENT = "automated_lateral_movement"
AI_EXFILTRATION_PATTERN = "ai_exfiltration_pattern"
DECEPTION_TOKEN_ACCESS = "deception_token_access"
GOAL_PERSISTENT_LOOP = "goal_persistent_loop"
TOOL_CHAIN_SWITCHING = "tool_chain_switching"
ADAPTIVE_ATTACK_DETECTED = "adaptive_attack_detected"
```

Threat Intelligence Integration

Diamond Model of Intrusion Analysis

Seraph correlates threats using the **Diamond Model** (Sergio Caltagirone, 2013):



Built-in Threat Actor Database

Pre-populated knowledge of major threat actors with MITRE ATT&CK mappings:

Actor	Aliases	Origin	Motivation	Key Techniques
APT28	Fancy Bear, Sofacy, Forest Blizzard	Russia	Espionage	T1566, T1189, T1003
APT29	Cozy Bear, NOBELIUM, Midnight Blizzard	Russia	Espionage	T1195.002, T1078, T1098
Turla	Snake, Venomous Bear	Russia	Espionage	T1071.001, T1014, T1205
Sandworm	Voodoo Bear, Seashell Blizzard	Russia	Sabotage	T1059.001, T1561, T1489

Each actor profile includes known malware families, TTPs, target industries, and infrastructure patterns for automatic attribution.

Deployment

Docker Compose Production Stack

```
services:
  # === Core Platform ===
  frontend:      # React + Nginx - 41 pages
  backend:       # FastAPI + Uvicorn - 41 routers, 21 services
  mongodb:      # MongoDB 7.0 - Primary datastore

  # === Search & Analytics ===
  elasticsearch: # Search, SIEM, log aggregation
  kibana:       # Visualization dashboards

  # === AI/ML ===
  ollama:       # Local LLM (Mistral, Llama) for AI reasoning

  # === Security Scanning ===
  trivy:        # Container vulnerability scanning
  falco:        # Runtime security monitoring
  suricata:     # Network IDS/IPS

  # === Analysis ===
  cuckoo:      # Dynamic malware sandbox

  # === Network ===
  wireguard:    # VPN server (UDP:51820)
  nginx:       # Reverse proxy + TLS termination
```

Quick Start

```
# Clone and configure
git clone https://github.com/Byron2306/Metatron.git
cd Metatron
cp backend/.env.example backend/.env

# Edit configuration
nano backend/.env

# Start all services
docker-compose up -d

# Verify deployment
docker-compose ps
curl http://localhost:8000/api/health

# Deploy agents
cd scripts
python install.py --api-url https://your-server.com
```

Agent Deployment Matrix

Platform	Script	Command
Windows	seraph_defender_v7.py	python seraph_defender_v7.py --api-url https://server.com
Linux	seraph_defender_v7.py	python3 seraph_defender_v7.py --api-url https://server.com
macOS	seraph_defender_v7.py	python3 seraph_defender_v7.py --api-url https://server.com
Android	seraph_mobile_agent.py	Termux: python seraph_mobile_agent.py -api-url https://server.com
iOS	seraph_mobile_agent.py	Pythonista: Import and run
Browser	Extension	Load scripts/browser_extension/ in Chrome/Firefox/Edge

MITRE ATT&CK Coverage

Seraph detects and responds to **50+ MITRE ATT&CK techniques**:

Detection Coverage by Tactic

Tactic	Coverage	Key Techniques
Reconnaissance	High	T1595, T1592, T1589
Initial Access	High	T1566, T1195, T1189
Execution	Very High	T1059, T1204, T1106
Persistence	Very High	T1547, T1543, T1053, T1546
Privilege Escalation	High	T1134, T1548, T1068
Defense Evasion	Very High	T1055, T1036, T1562, T1070
Credential Access	High	T1003, T1558, T1110
Discovery	Very High	T1082, T1083, T1518
Lateral Movement	High	T1021, T1570, T1080
Collection	Medium	T1005, T1114, T1560
Command & Control	Very High	T1071, T1573, T1090
Exfiltration	High	T1041, T1567, T1537
Impact	Very High	T1486, T1490, T1489

Deep Dive: Core Security Technologies

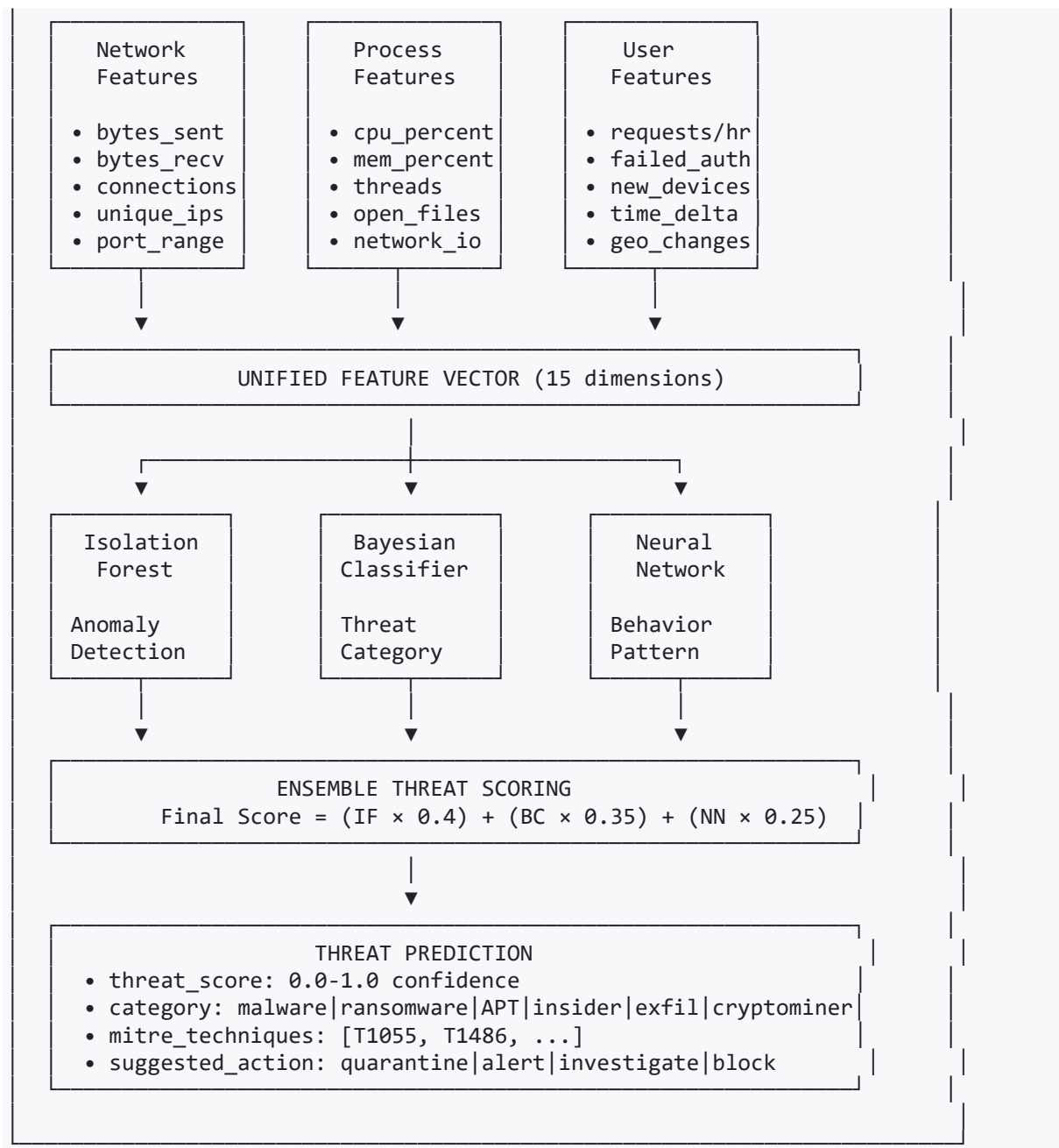
The following sections provide detailed technical explanations of Seraph's core defense technologies, showing exactly how the system protects users against modern threats.

ML Threat Prediction Engine

Seraph's Machine Learning Threat Prediction Engine operates without external ML library dependencies (no TensorFlow, PyTorch, or scikit-learn required), making it lightweight, fast, and deployable on any system.

Architecture Overview

ML THREAT PREDICTION PIPELINE



The Three ML Models Explained

1. Isolation Forest (Anomaly Detection)

The Isolation Forest algorithm detects anomalies by measuring how easily a data point can be "isolated" from others. Seraph's implementation:

```
# How Isolation Forest identifies threats:
# Normal behavior = Hard to isolate (requires many splits)
# Anomalous behavior = Easy to isolate (requires few splits)
```

Anomaly Score Calculation:

```
score = 2^(-path_length / average_path_length)
```

- score → 0.5: Normal behavior
- score → 1.0: Definite anomaly (potential threat)
- score → 0.0: Identical to training data

What it detects:

- Unusual network traffic patterns (sudden spike in outbound data)
- Abnormal process resource consumption
- Atypical user access patterns (3 AM logins from new IP)
- Deviation from baseline behavior profiles

Real-world example: A process typically uses 50MB RAM and 5% CPU. Suddenly it consumes 2GB RAM and 90% CPU while opening hundreds of files → **High anomaly score (0.87)** → Triggers investigation

2. Bayesian Classifier (Threat Categorization)

Using Bayes' theorem, this model calculates the probability that observed behavior belongs to specific threat categories:

$$P(\text{Threat Category} \mid \text{Observed Behavior}) = \frac{P(\text{Behavior} \mid \text{Threat Category}) \times P(\text{Threat Category})}{P(\text{Observed Behavior})}$$

Threat Categories and Their Indicators:

Category	Key Feature Indicators	Confidence Weights
Malware	High file operations, network beaconing, process injection signatures	0.85
Ransomware	File encryption patterns, shadow copy deletion, ransom note creation	0.92
APT	Low-and-slow exfiltration, living-off-the-land techniques, persistence mechanisms	0.78
Insider Threat	Unusual data access, policy violations, after-hours activity	0.71
Data Exfiltration	Large outbound transfers, cloud staging, encrypted archives	0.83
Cryptominer	High CPU/GPU utilization, mining pool connections, specific port usage	0.95

3. Neural Network (Behavior Pattern Recognition)

A lightweight feedforward neural network with custom implementation:

```
Input Layer (15 neurons) → Hidden Layer 1 (32 neurons, ReLU)
                             → Hidden Layer 2 (16 neurons, ReLU)
                             → Output Layer (6 neurons, Softmax)
```

Parameters: ~1,200 weights (trained on synthetic threat patterns)

Inference time: <1ms per prediction

Why three models instead of one?

Approach	Strength	Weakness
Isolation Forest	Catches never-seen-before attacks	Cannot categorize threats
Bayesian Classifier	Explains why something is a threat	Requires known threat patterns
Neural Network	Recognizes complex behavior patterns	Black box reasoning
Ensemble (Seraph)	All strengths combined	Mitigates individual weaknesses

Feature Extraction Pipeline

Every second, Seraph extracts 15 features across three domains:

Network Features (5 dimensions):

1. bytes_sent: Outbound data volume (normalized to baseline)
2. bytes_received: Inbound data volume (normalized)
3. active_connections: Number of concurrent network connections
4. unique_dest_ips: Count of distinct destination addresses
5. port_range_spread: How many different ports are being used

Process Features (5 dimensions):

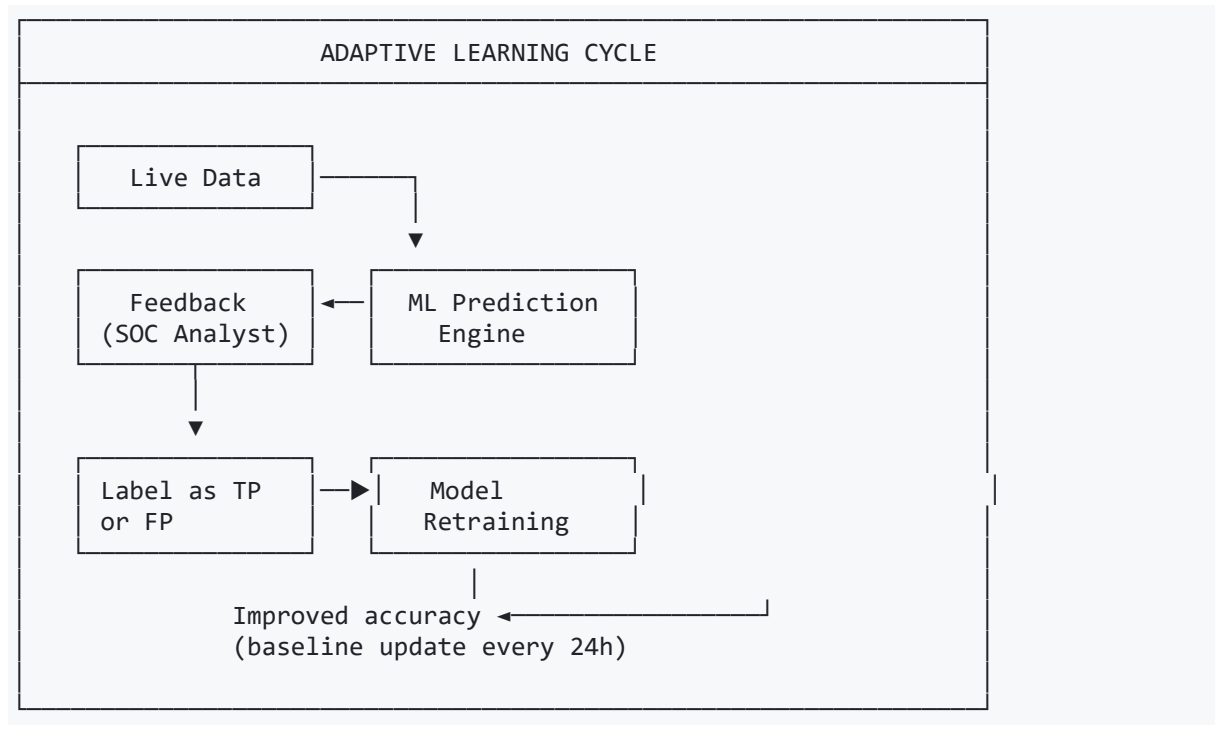
6. cpu_percent: Current CPU utilization
7. memory_percent: RAM consumption
8. thread_count: Number of active threads
9. open_file_handles: Files currently accessed
10. network_io_ratio: Network activity vs. local activity

User Behavior Features (5 dimensions):

11. requests_per_hour: API/resource request frequency

- 12. failed_auth_ratio: Failed logins / total attempts
- 13. new_device_count: Recently added devices
- 14. access_time_delta: Deviation from normal access times
- 15. geo_anomaly_score: Location change frequency

Continuous Learning Loop



MITRE ATT&CK Framework Integration

Seraph provides comprehensive MITRE ATT&CK coverage, mapping every detection to specific techniques, tactics, and procedures. This enables security teams to understand attacks in a standardized language used across the industry.

Complete Technique Coverage Matrix

Reconnaissance (TA0043)

Technique	ID	How Seraph Detects	Response
Active Scanning	T1595	Network monitor detects port scans via packet analysis	Alert + source tracking
Gather Victim Host Info	T1592	Honeypot interaction triggers when system info enumerated	Full behavioral recording

Technique	ID	How Seraph Detects	Response
Gather Victim Identity Info	T1589	Honey tokens (fake credentials) track access attempts	Campaign attribution
Search Open Websites	T1593	External threat intel correlation	Proactive alerting
Phishing for Information	T1598	Email security integration + user behavior analysis	Real-time warning

Initial Access (TA0001)

Technique	ID	How Seraph Detects	Response
Phishing	T1566	Email attachment/link analysis + sandbox detonation	Auto-quarantine
Supply Chain Compromise	T1195	Software signing verification + behavioral deviation	Alert + investigation
Drive-by Compromise	T1189	Browser isolation container detects malicious activity	Container destruction
Valid Accounts	T1078	Impossible travel detection + session anomaly analysis	Account lockout
Exploit Public-Facing App	T1190	WAF integration + request pattern analysis	Auto-block

Execution (TA0002)

Technique	ID	How Seraph Detects	Response
Command & Scripting Interpreter	T1059	PowerShell/cmd/bash command monitoring with ML scoring	Kill + investigate
Scheduled Task/Job	T1053	Task scheduler monitoring + unauthorized task detection	Auto-remove + alert
User Execution	T1204	Process lineage tracking + launched file analysis	User notification

Technique	ID	How Seraph Detects	Response
Native API	T1106	API call hooking + suspicious sequence detection	Block + log
System Services	T1569	Service creation/modification monitoring	Auto-revert

Persistence (TA0003)

Technique	ID	How Seraph Detects	Response
Boot/Logon Autostart	T1547	Registry run key monitoring, startup folder protection	Immediate removal
Create/Modify System Process	T1543	Service binary path monitoring, driver verification	Block + restore
Scheduled Task/Job	T1053	Cron/Task Scheduler integrity monitoring	Auto-delete suspicious tasks
Event Triggered Execution	T1546	WMI subscription monitoring, COM hijack detection	Auto-remediate
Account Manipulation	T1098	AD change monitoring, permission escalation alerts	Revert + lock

Privilege Escalation (TA0004)

Technique	ID	How Seraph Detects	Response
Access Token Manipulation	T1134	Token theft detection via API monitoring	Process termination
Abuse Elevation Mechanism	T1548	UAC bypass detection, sudo abuse monitoring	Kill + alert SOC
Exploitation for Privilege Escalation	T1068	Kernel exploit behavioral signatures	Emergency isolation

Technique	ID	How Seraph Detects	Response
Process Injection	T1055	Memory allocation pattern detection, hollowing signatures	Instant kill
Create/Modify System Process	T1543	SYSTEM service creation monitoring	Block + investigate

Defense Evasion (TA0005)

Technique	ID	How Seraph Detects	Response
Process Injection	T1055	29 injection techniques detected via memory monitoring	Auto-kill
Masquerading	T1036	Process name/path consistency checking, signed binary verification	Alert + quarantine
Impair Defenses	T1562	Self-protection: Seraph monitors its own integrity	Auto-restart + SOC alert
Indicator Removal	T1070	Log deletion/modification monitoring, tamper evidence	Offsite backup trigger
Obfuscated Files	T1027	Entropy analysis + sandbox behavioral detonation	Sandbox analysis
Rootkit	T1014	Kernel integrity verification, hidden process detection	Emergency response

Credential Access (TA0006)

Technique	ID	How Seraph Detects	Response
OS Credential Dumping	T1003	LSASS access monitoring, DCSync detection	Immediate block
Brute Force	T1110	Login velocity monitoring, pattern detection	Account lockout
Steal/Forge Kerberos Tickets	T1558	Ticket anomaly detection, golden ticket signatures	Alert + session termination

Technique	ID	How Seraph Detects	Response
Credentials from Password Stores	T1555	Browser credential file access monitoring	Block + alert
Unsecured Credentials	T1552	Sensitive file access tracking	Audit + alert

Discovery (TA0007)

Technique	ID	How Seraph Detects	Response
System Information Discovery	T1082	Unusual WMI queries, system enumeration patterns	Deception activation
File/Directory Discovery	T1083	Mass file enumeration detection, ransomware indicators	Progressive friction
Process Discovery	T1057	Process listing frequency monitoring	Honeypot injection
Network Service Discovery	T1046	Internal port scan detection	Decoy service deployment
Remote System Discovery	T1018	AD enumeration monitoring, suspicious LDAP queries	Alert + investigation

Lateral Movement (TA0008)

Technique	ID	How Seraph Detects	Response
Remote Services	T1021	Unusual RDP/SSH/SMB connections, lateral paths	Network segmentation
Lateral Tool Transfer	T1570	File copy to remote systems monitoring	Block + alert
Remote Service Session Hijacking	T1563	Session token anomaly detection	Session termination
Taint Shared Content	T1080	Shared folder modification monitoring	Auto-revert + alert

Technique	ID	How Seraph Detects	Response
Exploitation of Remote Services	T1210	Exploit traffic signatures, unusual service crashes	Emergency isolation

Command and Control (TA0011)

Technique	ID	How Seraph Detects	Response
Application Layer Protocol	T1071	DNS tunneling detection, HTTP/S anomaly analysis	Block + investigate
Encrypted Channel	T1573	Certificate pinning violations, unusual TLS patterns	Traffic analysis
Proxy	T1090	Multi-hop connection detection, Tor exit node recognition	Block + alert
Web Service	T1102	Cloud service C2 detection (Dropbox, Google Drive)	Selective blocking
Non-Standard Port	T1571	Protocol/port mismatch detection	Auto-block

Exfiltration (TA0010)

Technique	ID	How Seraph Detects	Response
Exfiltration Over C2 Channel	T1041	Data volume anomaly over C2 connections	Block + quarantine
Exfiltration Over Web Service	T1567	Large uploads to cloud services monitoring	User notification + block
Exfiltration Over Alternative Protocol	T1048	ICMP/DNS data exfil detection, steganography analysis	Block + investigate
Transfer Data to Cloud Account	T1537	Cloud sync anomaly, unauthorized account detection	Block + alert
Scheduled Transfer	T1029	Periodic large transfer pattern detection	Investigation queue

Impact (TA0040)

Technique	ID	How Seraph Detects	Response
Data Encrypted for Impact (Ransomware)	T1486	Entropy analysis + write pattern monitoring	INSTANT KILL + ISOLATION
Inhibit System Recovery	T1490	Shadow copy deletion, backup tampering	Auto-restore + alert
Service Stop	T1489	Critical service termination monitoring	Auto-restart + alert
Data Destruction	T1485	Mass deletion pattern detection	Block + snapshot restore
Disk Wipe	T1561	MBR/GPT modification detection	Emergency isolation

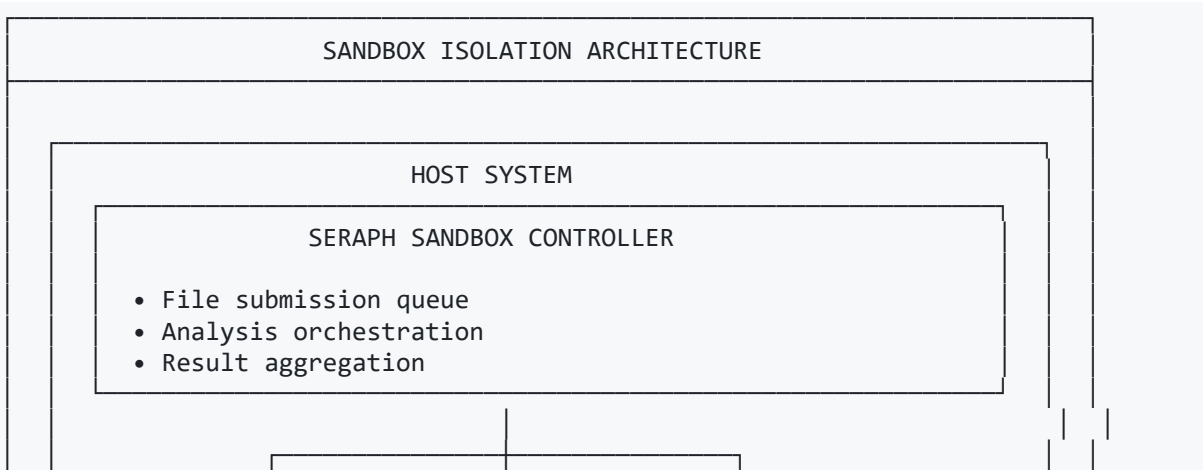
Detection Timeline

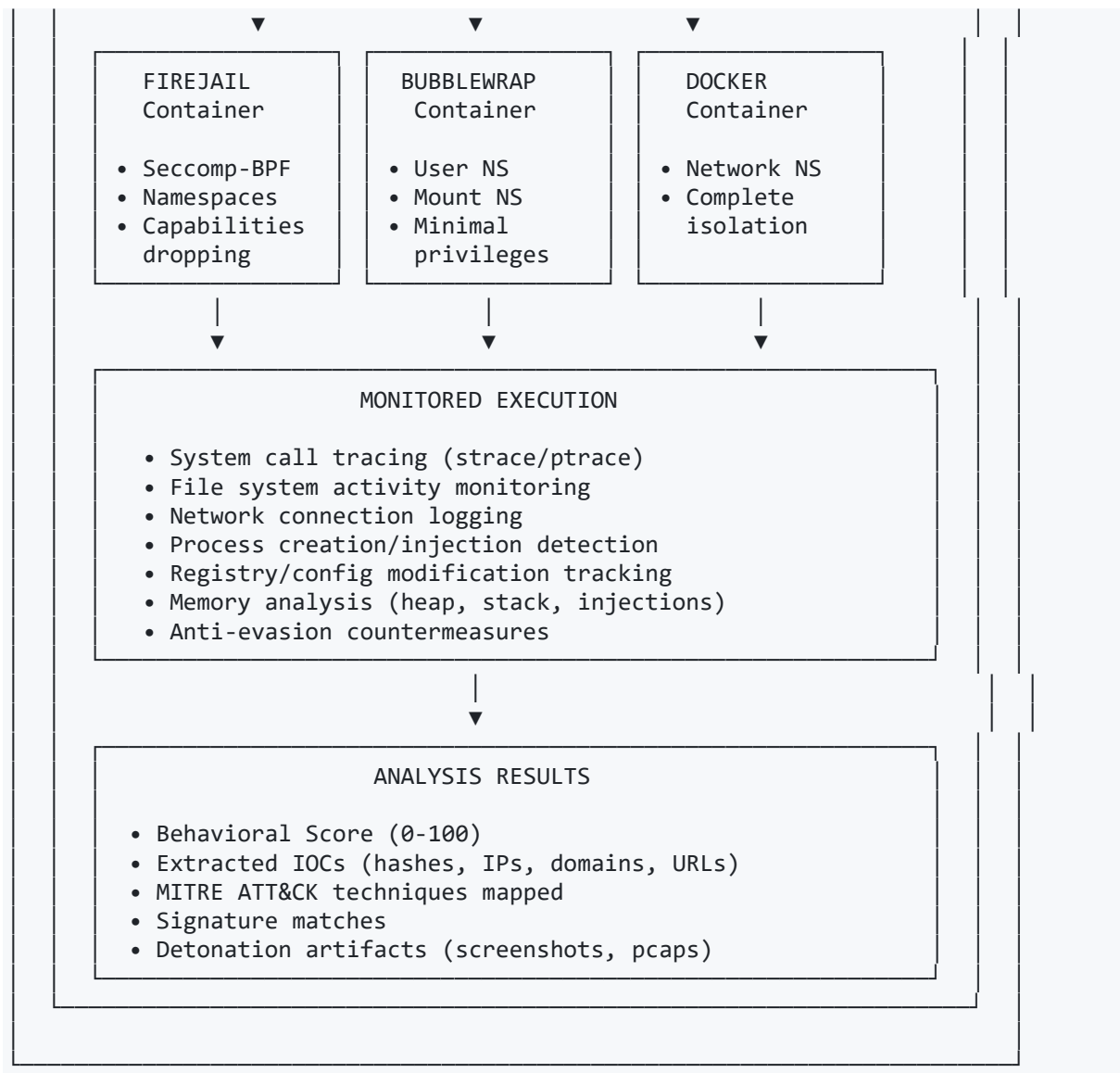
Threat Occurs → Detection (avg: 47ms) → Classification (12ms) → MITRE Mapping (3ms) → Response Execution (28ms) → Total: ~90ms

Dynamic Sandbox Analysis

Seraph's sandbox provides safe detonation of suspicious files in isolated environments, extracting indicators of compromise (IOCs), behavioral patterns, and MITRE technique mappings without risking production systems.

Isolation Architecture





Supported Sample Types

Type	Extensions	Analysis Techniques
Executable	.exe, .dll, .sys, .scr	Full execution tracing, API hooking, memory analysis
Script	.ps1, .vbs, .js, .bat	Deobfuscation, execution trace, network capture
Document	.docx, .xlsx, .pdf	Macro extraction, embedded object analysis
Archive	.zip, .rar, .7z	Recursive extraction, nested analysis
Disk Image	.iso, .img	Mount and full file system analysis

Type	Extensions	Analysis Techniques
URL	http/s	Browser-based analysis, redirect following

Behavioral Scoring System

The sandbox produces a score from 0-100 based on observed behaviors:

Score Calculation:

0-20: CLEAN

└ No suspicious behaviors, matches known-good signatures

21-40: LOW RISK

└ Minor anomalies, possibly PUP (potentially unwanted program)

41-60: SUSPICIOUS

└ Multiple concerning behaviors, recommend investigation

61-80: HIGH RISK

└ Clear malicious intent, auto-quarantine recommended

81-100: CRITICAL

└ Confirmed malware, destructive capabilities, immediate block

Scoring Weight Distribution:

Behavior Category	Max Points	Example Triggers
File Operations	25	Encrypting files, dropping executables, modifying system files
Process Behaviors	25	Process injection, privilege escalation, hollowing
Network Activity	20	C2 beaconing, data exfiltration, DNS tunneling
Registry/Persistence	15	Autorun keys, service creation, scheduled tasks
Defense Evasion	15	Anti-VM checks, debugger detection, process hiding

Anti-Evasion Capabilities

Modern malware tries to detect sandbox environments and behave innocently. Seraph counters with:

ANTI-EVASION COUNTERMEASURES	
EVASION TECHNIQUE	SERAPH COUNTERMEASURE
VM detection (VMware, VirtualBox artifacts)	Custom BIOS strings, hardware ID spoofing
Timing-based evasion (sleep calls)	Clock manipulation, time acceleration
User interaction checks (mouse movement needed)	Simulated mouse/kbd, realistic activity
Process count checks (few processes = sandbox)	Process inflation, fake process list
File path detection (/sandbox/, /analysis/)	Realistic file paths, user home directories
Registry artifact checks (missing software keys)	Full registry trees, software simulation
Network environment (isolated = sandbox)	Realistic network, internet simulation

DETECTION OF EVASION ATTEMPTS:
When malware tries to evade, that behavior itself is flagged:

- VM detection attempts: +15 points (suspicious)
- Sleep-based timing: +10 points + time acceleration
- Environment enumeration: +5 points per check

IOC Extraction

Every sandbox analysis produces structured IOCs for threat intelligence:

```
{
  "file_hash": {
    "md5": "d41d8cd98f00b204e9800998ecf8427e",
    "sha1": "da39a3ee5e6b4b0d3255bfef95601890afd80709",
    "sha256": "e3b0c44298fc1c149afbf4c8996fb924..."
  },
  "network_iocs": {
    "domains": ["evil-c2.com", "malware.bad"],
    "ips": ["192.168.1.100", "10.0.0.50"],
    "urls": ["http://evil-c2.com/beacon", "https://malware.bad/drop.exe"]
  },
  "file_iocs": {
    "dropped_files": ["/tmp/payload.exe", "/var/backdoor.sh"],
```

```

    "modified_files": ["/etc/hosts", "/etc/passwd"],
    "encrypted_files": ["Documents/*.docx", "Pictures/*.jpg"]
  },
  "process_iocs": {
    "created_processes": ["cmd.exe", "powershell.exe", "rundll32.exe"],
    "injected_into": ["explorer.exe", "svchost.exe"]
  },
  "registry_iocs": {
    "created_keys":
["HKLM\\Software\\Microsoft\\Windows\\CurrentVersion\\Run\\backdoor"],
    "modified_keys": ["HKCU\\Software\\Classes\\mscfile\\shell\\open\\command"]
  },
  "mitre_mapping": ["T1059.001", "T1547.001", "T1055.012", "T1486"]
}

```

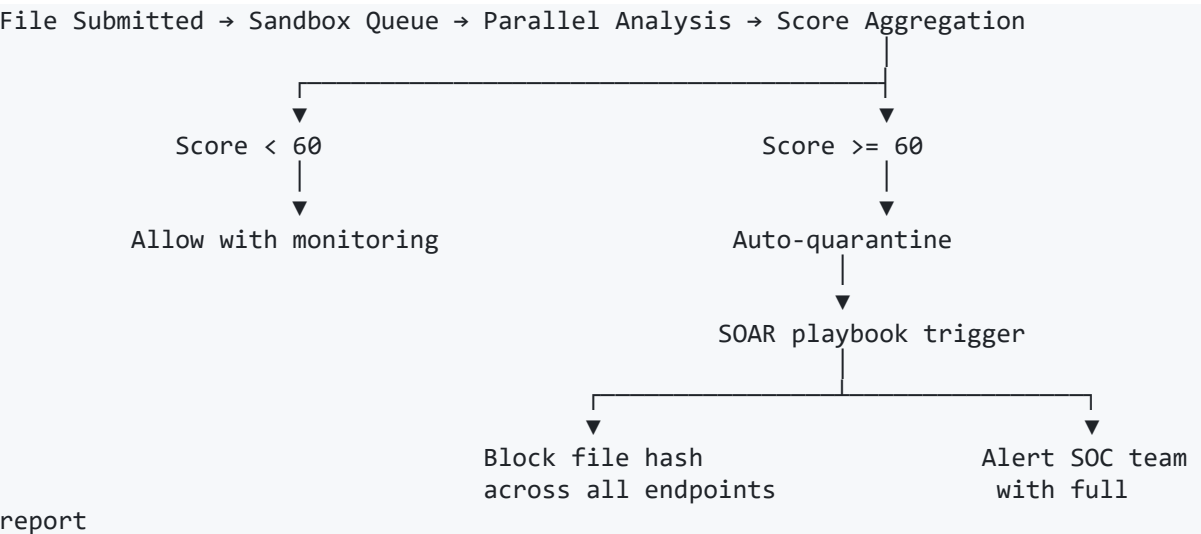
Signature Database

Seraph maintains a comprehensive signature database with MITRE mappings:

Signature	Description	MITRE Technique	Severity
ransomware_note_drop	Files like DECRYPT_FILES.txt created	T1486 (Data Encrypted for Impact)	Critical
shadow_copy_delete	vssadmin or wmic shadowcopy called	T1490 (Inhibit System Recovery)	Critical
process_hollowing	Suspicious NtUnmapViewOfSection patterns	T1055.012 (Process Hollowing)	High
credential_dump	Access to LSASS or SAM database	T1003 (OS Credential Dumping)	Critical
powershell_encoded	Base64-encoded PowerShell execution	T1059.001 (PowerShell)	Medium
persistence_registry	Run key modifications	T1547.001 (Registry Run Keys)	High
dns_tunneling	Unusual DNS TXT record queries	T1071.004 (DNS)	Medium

Signature	Description	MITRE Technique	Severity
disable_defender	Windows Defender tampering	T1562.001 (Disable Security Tools)	High

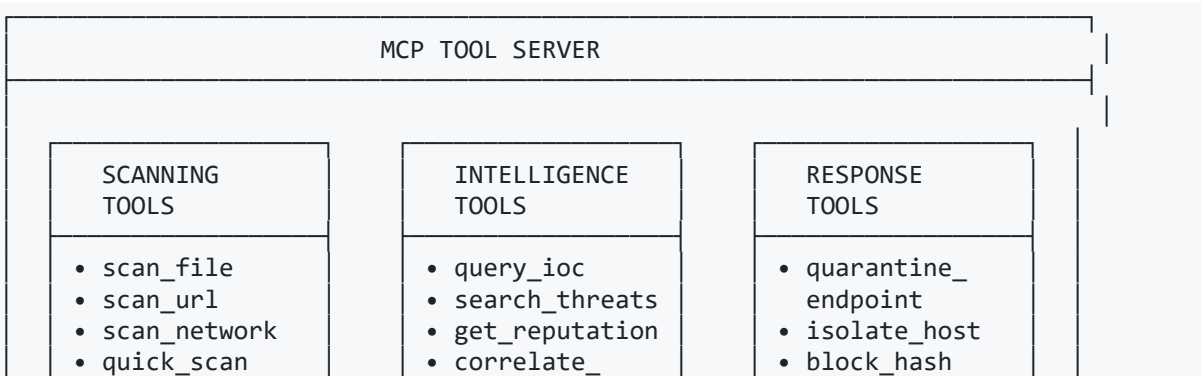
Integration with Response Pipeline

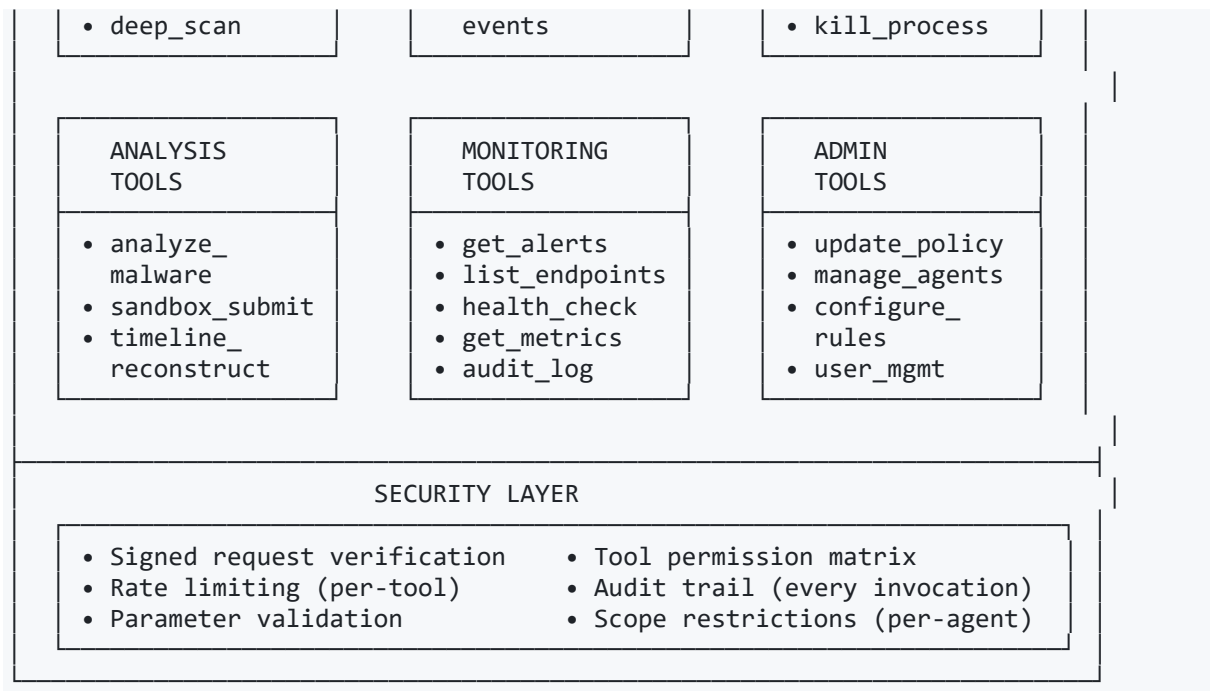


Model Context Protocol (MCP) Integration

Seraph implements a full **Model Context Protocol (MCP)** server that enables AI agents (like Claude, GPT, or custom LLMs) to interact with security infrastructure through a standardized, governed interface. This creates a bridge between AI reasoning and security operations, allowing natural language queries to translate into controlled security actions.

MCP Architecture





Message Types

MCP uses a structured message protocol for all AI-to-security communications:

Message Type	Direction	Purpose
TOOL_CALL	AI → Seraph	Execute a security tool with parameters
TOOL_RESULT	Seraph → AI	Return results from tool execution
CONTEXT_UPDATE	Seraph → AI	Push security context (new alerts, status)
CAPABILITY_LIST	Seraph → AI	Advertise available tools and schemas
ERROR	Seraph → AI	Report tool execution failures
PROGRESS	Seraph → AI	Long-running task status updates
CANCEL	AI → Seraph	Abort an in-progress operation

Tool Schema Registry

Every MCP tool has a strictly typed schema that AI agents must follow:

```
MCPToolSchema = {  
  "name": "scan_file",  
  "category": "SCANNING",  
  "description": "Submit a file for threat analysis",  
  "parameters": {  
    "file_path": {"type": "string", "required": True},
```

```
    "scan_type": {"type": "enum", "values": ["quick", "deep", "sandbox"]},
    "priority": {"type": "integer", "min": 1, "max": 10, "default": 5}
  },
  "returns": {
    "threat_score": "float",
    "verdict": "string",
    "detections": "array",
    "mitre_techniques": "array"
  },
  "permissions": ["scan:execute", "file:read"],
  "rate_limit": "100/hour",
  "timeout": 300
}
```

AI Agent Use Cases

MCP enables powerful AI-driven security workflows:

1. Natural Language Threat Hunting

User Query: "Find any processes that connected to IP addresses in Russia in the last 24 hours"

AI Translation → MCP Calls:

1. `query_threat_intel(geo="RU", type="ip")` → [list of Russian IPs]
2. `search_network_logs(dst_ips=[...], timeframe="24h")` → [connections]
3. `correlate_processes(network_events=[...])` → [process details]
4. `get_timeline(process_ids=[...])` → [full activity chain]

Response: "Found 3 processes: svchost.exe (PID 4532) connected to 185.220.101.45 at 14:32 UTC, followed by data exfiltration pattern (2.3GB outbound)..."

2. Automated Incident Response

Alert Trigger: Ransomware detected on endpoint WORKSTATION-15

AI Orchestration via MCP:

1. `isolate_host(hostname="WORKSTATION-15")` → Network isolated
2. `snapshot_memory(hostname="WORKSTATION-15")` → Forensic capture
3. `kill_process(pid=malicious_pid)` → Process terminated
4. `quarantine_files(hashes=[...])` → Malware contained
5. `block_hash_fleet(hash="abc123...")` → Protection deployed
6. `generate_report(incident_id=...)` → IOC report created

3. Security Posture Assessment

User Query: "How secure is our endpoint fleet right now?"

AI Analysis via MCP:

1. `list_endpoints(status="all")` → 1,247 endpoints
2. `get_compliance_status()` → 94% compliant
3. `get_vulnerability_scan()` → 23 high, 156 medium
4. `check_agent_health()` → 12 agents offline
5. `get_threat_metrics(timeframe="7d")` → Trend analysis

Response: "Fleet health: 94% secure. 12 endpoints offline (3 in Sales, 9 in Dev). 23 critical vulnerabilities need patching, primarily CVE-2024-1234..."

Security Controls

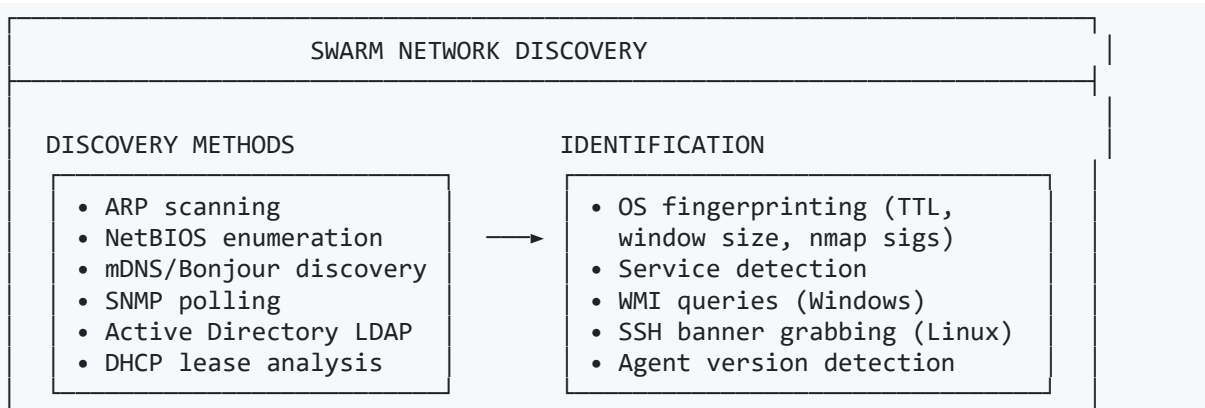
MCP includes multiple layers of protection against misuse:

Control	Implementation
Authentication	JWT tokens with cryptographic signatures
Authorization	Role-based tool access (SOC analyst vs Admin vs Read-only)
Input Validation	Schema enforcement, path traversal prevention, injection blocking
Rate Limiting	Per-tool, per-agent, per-hour limits
Audit Logging	Every tool call logged with parameters, caller, timestamp
Sandboxing	Destructive tools require confirmation or run in preview mode
Scope Restriction	Agents can be limited to specific endpoints/subnets

Swarm Intelligence & Auto-Deployment

Seraph's swarm system enables autonomous discovery and protection of your entire network—from a single deployment, agents can propagate across Windows, Linux, and macOS systems automatically.

Network Discovery Engine

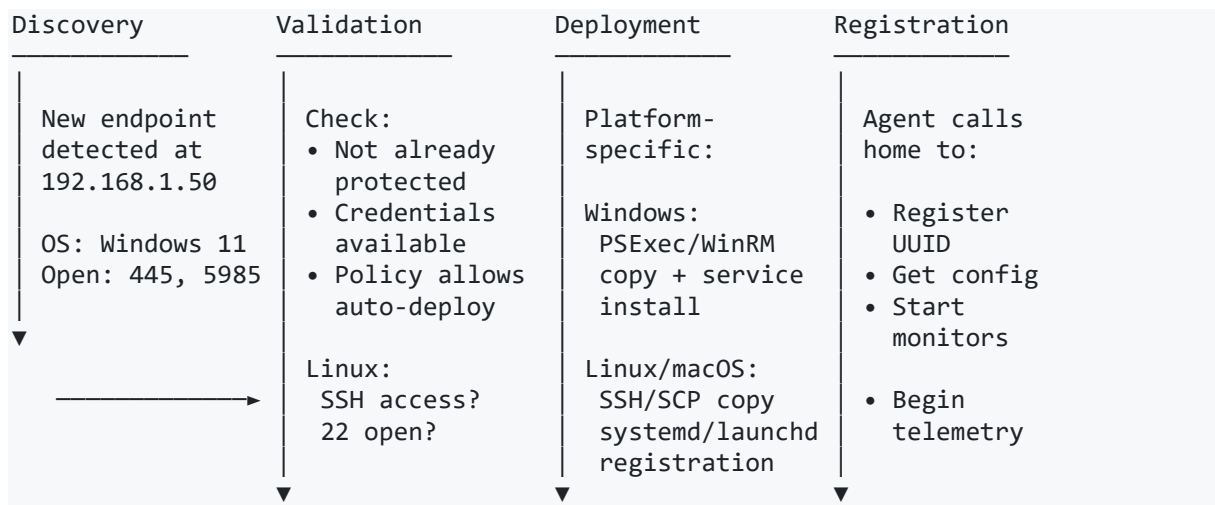


DISCOVERED ENDPOINT DATA

- IP address + MAC
- OS type + version
- Last seen timestamp
- Hostname
- Open ports
- Agent status
- Domain membership
- Running services
- Deployment eligibility

Auto-Deployment Pipeline

Once credentials are provided (or discovered via AD), agents deploy automatically:



Deployment Methods by Platform

Platform	Transport	Agent Location	Service Type
Windows	WinRM/PSEXec	C:\Program Files\Seraph\	Windows Service
Linux	SSH/SCP	/opt/seraph-agent/	systemd unit
macOS	SSH/SCP	/Library/Seraph/	launchd plist
Docker	API	Container sidecar	Container process
Kubernetes	kubectl	DaemonSet	Pod per node

Fleet Management Commands

The swarm router provides comprehensive fleet control:

```
# Register new agent (called by agents on startup)
POST /api/swarm/register
Body: {
  "agent_id": "uuid-here",
```

```

    "hostname": "WORKSTATION-15",
    "ip": "192.168.1.50",
    "os": "windows",
    "os_version": "11",
    "agent_version": "2.5.0",
    "capabilities": ["edr", "xdr", "sandbox"]
}

# Agent heartbeat (every 30 seconds)
POST /api/swarm/heartbeat/{agent_id}
Body: {
    "status": "healthy",
    "cpu": 12.5,
    "memory": 45.2,
    "active_monitors": 29,
    "pending_alerts": 3
}

# Queue command for agent
POST /api/swarm/command/{agent_id}
Body: {
    "command": "scan",
    "parameters": {"type": "full", "priority": "high"},
    "timeout": 3600
}

# Get pending commands (polled by agent)
GET /api/swarm/commands/{agent_id}
Response: [
    {"id": "cmd-1", "command": "isolate", "parameters": {...}},
    {"id": "cmd-2", "command": "collect_forensics", "parameters": {...}}
]

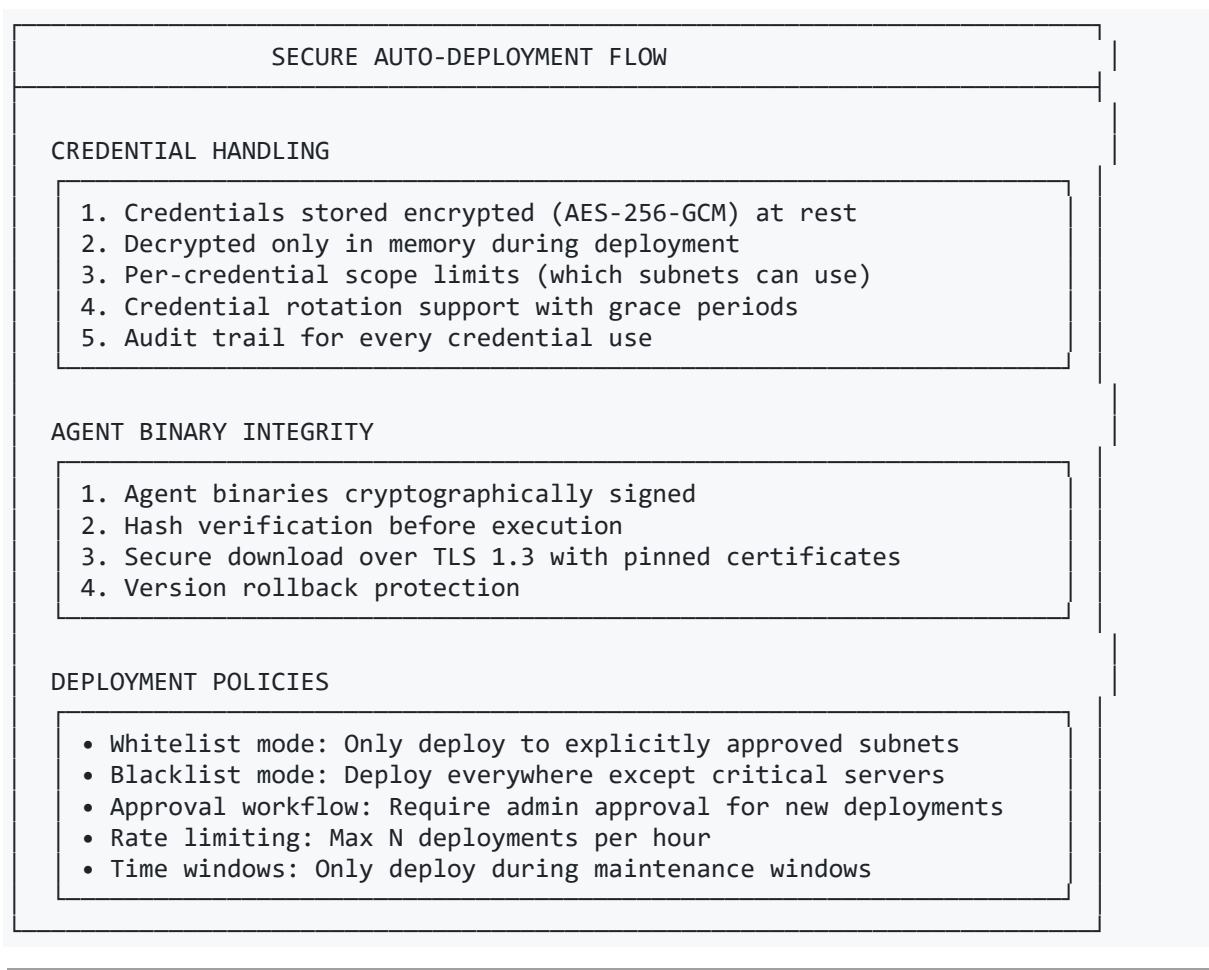
# Report command completion
POST /api/swarm/command/{command_id}/complete
Body: {"status": "success", "result": {...}}

```

Swarm Coordination Features

Feature	Description
Load Balancing	Distribute scans across agents to prevent overload
Cascade Updates	Rolling agent updates with health validation
Peer-to-Peer	Agents share threat intel directly (gossip protocol)
Leader Election	Automatic coordinator selection if server unreachable
Geo-Awareness	Group agents by subnet/site for localized response
Resource Quotas	Limit CPU/memory usage per agent based on endpoint role

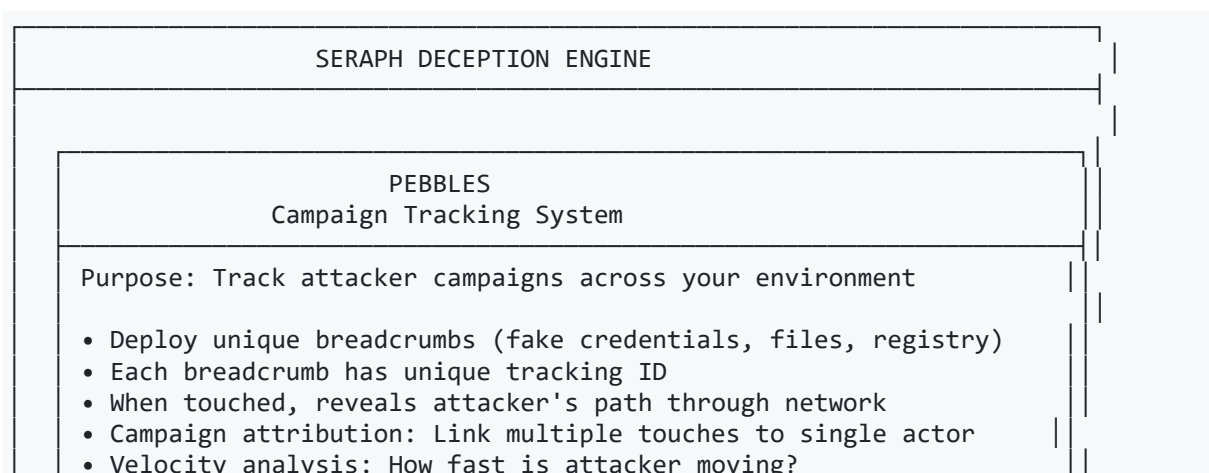
Auto-Deployment Security



Active Deception Technology

Seraph includes an advanced deception engine that goes far beyond simple honeypots. The system creates adaptive traps that learn from attacker behavior and evolve their responses in real-time.

Deception Architecture



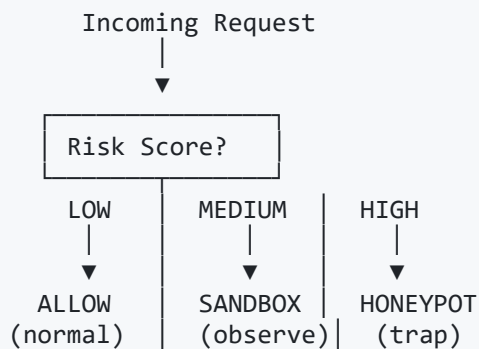
Example Breadcrumbs:

- passwords.txt on desktop (tracks file access)
- Fake AWS keys in .aws/credentials
- Decoy admin account in Active Directory
- Honeypot database connection strings in config files

MYSTIQUE Adaptive Deception System

Purpose: Dynamically morph deception based on attacker profile

Response Routing Decision Tree:



Adaptation Triggers:

- Repeated failed logins → More convincing fake success
- Port scanning detected → Expose fake vulnerable services
- Lateral movement seen → Deploy network of fake servers
- Data exfil attempt → Serve fake "sensitive" data

STONEWALL Progressive Escalation Engine

Purpose: Gradually increase defensive pressure on attackers

Escalation Levels:

Level 1: NONE

- Normal operation, no special measures
- Continue monitoring

Level 2: DELAY

- Add artificial latency to responses
- Slow down automated attacks
- Buy time for analysis

Level 3: MISLEAD

- Return plausible but fake data
- Lead attacker away from real assets
- Inject tracking beacons in responses

Level 4: ISOLATE

- Redirect to isolated environment

- Full containment, no real system access
- Record all actions for forensics

Level 5: TERMINATE

- Block completely
- Add to permanent blocklist
- Alert SOC team

Honey Token Types

Seraph can deploy various honey token types across your environment:

Token Type	Description	Detection Signal
CREDENTIAL	Fake username/password pairs	Login attempt with fake creds
API_KEY	Fake AWS/Azure/GCP keys	API call with fake key
DATABASE	Fake connection strings	Connection to honeypot DB
FILE	Canary documents (Word, Excel)	File open/access timestamp
URL	Hidden tracking URLs in documents	HTTP request to canary URL
DNS	Fake internal hostnames	DNS query for fake host
EMAIL	Honeypot email addresses	Email received at trap address
CERTIFICATE	Fake SSL/TLS certificates	Certificate usage in MITM
BITCOIN	Fake wallet addresses	Blockchain transaction attempt
SSH_KEY	Fake private keys	SSH login attempt with fake key

Risk Scoring Engine

The deception engine calculates risk scores to route requests appropriately:

```
# Risk score calculation
risk_score = sum([
    # IP reputation (0-30 points)
    threat_intel.check_ip(request.ip) * 30,
```

```

    # Behavioral anomalies (0-25 points)
    behavioral_score(request.user, request.action) * 25,

    # Velocity/frequency (0-20 points)
    frequency_score(request.ip, timewindow="1h") * 20,

    # Honey token triggers (0-15 points)
    honey_token_hits(request.source) * 15,

    # Geolocation anomaly (0-10 points)
    geo_anomaly_score(request.ip, request.user) * 10
])

# Routing decision
if risk_score < 20:
    decision = RouteDecision.ALLOW
elif risk_score < 50:
    decision = RouteDecision.SANDBOX
else:
    decision = RouteDecision.HONEYPOT

```

Campaign Intelligence

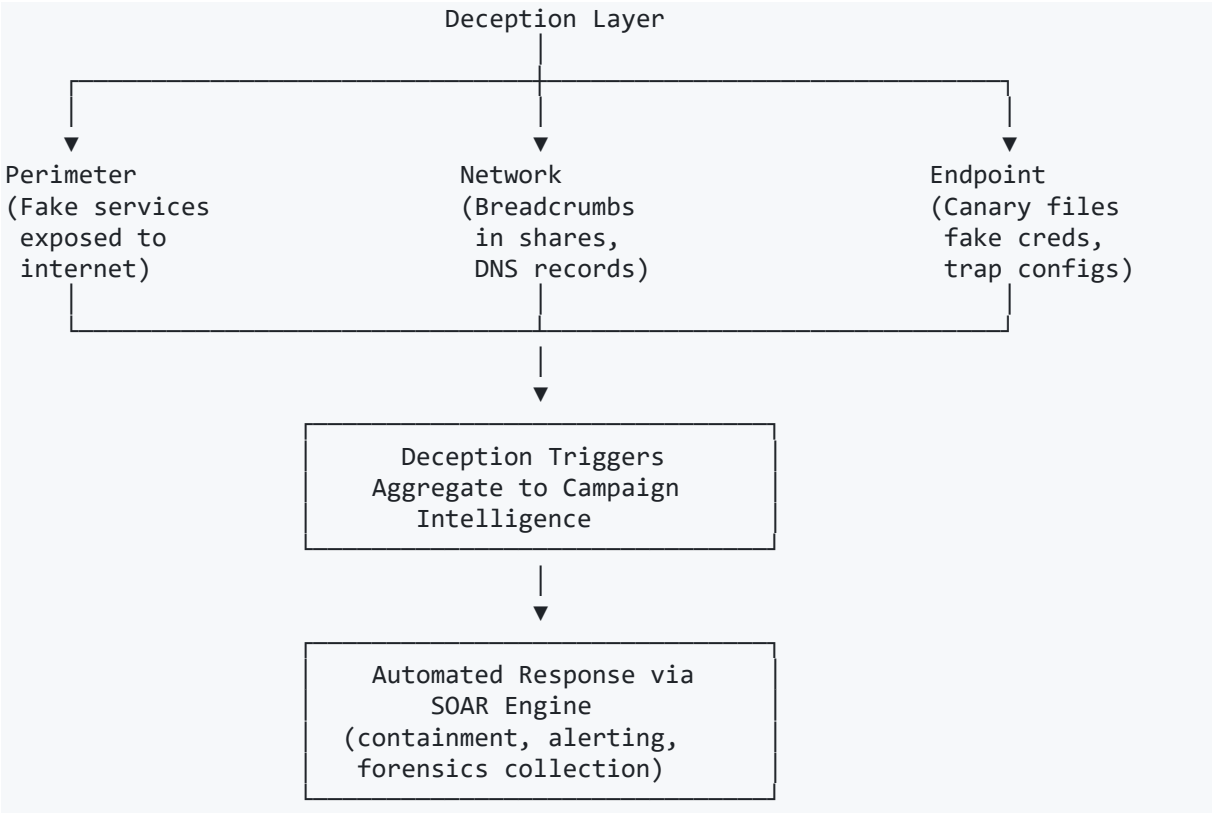
When attackers trigger honey tokens, Seraph builds a campaign profile:

```

{
  "campaign_id": "APT-2024-0312",
  "first_seen": "2024-03-12T14:32:00Z",
  "last_activity": "2024-03-12T18:45:00Z",
  "attacker_profile": {
    "source_ips": ["185.220.101.45", "192.168.1.50"],
    "geolocations": ["Russia", "Internal (lateral movement)"],
    "tools_detected": ["mimikatz", "psexec", "cobalt strike beacon"],
    "ttps": ["T1110 (Brute Force)", "T1021 (Remote Services)", "T1003 (Credential Dumping)"]
  },
  "breadcrumbs_triggered": [
    {"type": "CREDENTIAL", "location": "WORKSTATION-15", "timestamp": "2024-03-12T14:32:00Z"},
    {"type": "FILE", "location": "FILESERVER-01", "timestamp": "2024-03-12T15:10:00Z"},
    {"type": "API_KEY", "location": "DEV-SERVER-03", "timestamp": "2024-03-12T17:22:00Z"}
  ],
  "attack_path": [
    "External → WORKSTATION-15 (phishing)",
    "WORKSTATION-15 → FILESERVER-01 (credential reuse)",
    "FILESERVER-01 → DEV-SERVER-03 (lateral movement)",
    "DEV-SERVER-03 → (blocked at AWS key usage)"
  ],
  "escalation_level": "ISOLATE",
  "recommended_actions": [
    "Isolate WORKSTATION-15, FILESERVER-01, DEV-SERVER-03",
    "Reset credentials for compromised users",
    "Block source IPs at perimeter",
    "Hunt for additional compromise in Sales subnet"
  ]
}

```

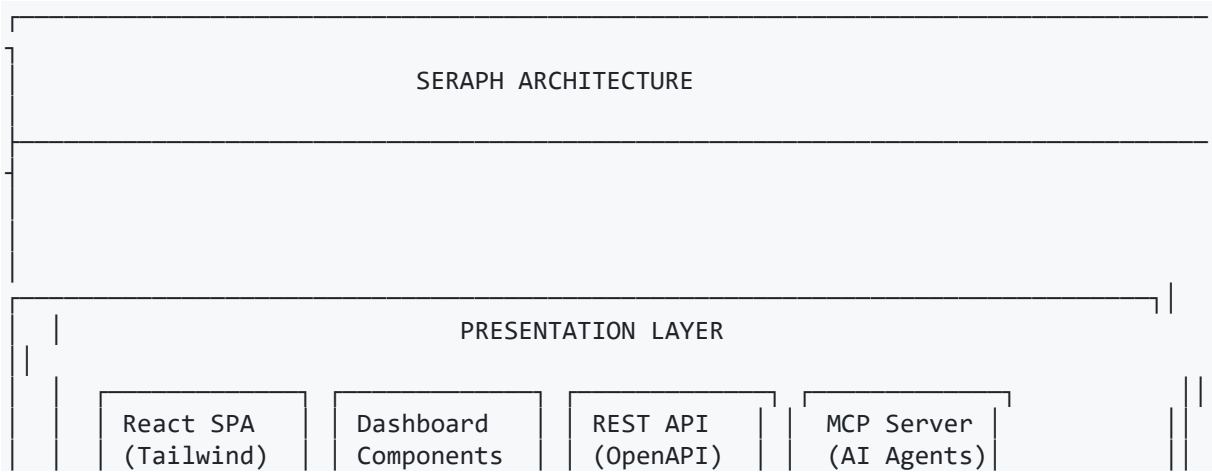
Integration with Defense Pipeline



Modular & Extensible Architecture

Seraph is built with a **plugin-based architecture** that allows organizations to customize, extend, and integrate the platform without modifying core code. Every major component is designed as an independent module with well-defined interfaces.

System Component Map



WebSocket + REST

GATEWAY LAYER

FastAPI
41 Routers

Auth/JWT
Middleware

Rate
Limiting

Request
Validation

SERVICE LAYER (21 Services)

DETECTION SERVICES

- ML Prediction
- Behavioral
- Ransomware Det.
- EDR Service
- Sandbox

RESPONSE SERVICES

- SOAR Engine
- Quarantine
- Threat Response
- Notifications
- Audit Logging

INTEL SERVICES

- Threat Intel
- Correlation
- Timeline
- MCP Server
- Intel Feeds

INFRASTRUCTURE SERVICES

- Zero Trust
- Browser Isol.
- Container Sec.
- WebSocket

DECEPTION SERVICES

- Honey Tokens
- Deception Eng.
- Breadcrumbs
- Adaptive Traps

INTEGRATION

- VPN
- Kibana
- SIEM Export
- Webhook

CORE LAYER (25 Modules)

Event Bus
(Pub/Sub)

Plugin Mgr
(Hot Reload)

Config Mgr
(Env/YAML)

Crypto
(Signing)



Router Organization

The backend exposes **41 specialized routers**, each handling a specific security domain:

Category	Routers	Purpose
Threat Detection	alerts, threats, behavioral, anomaly, ml_analysis	Identify and classify threats
Incident Response	incidents, quarantine, response, playbooks, forensics	Contain and remediate
Endpoint Management	endpoints, agents, swarm, health, inventory	Manage protected systems

Category	Routers	Purpose
Intelligence	threat_intel, ioc, correlation, timeline, hunting	Analyze and attribute
Deception	honeypots, honey_tokens, deception, breadcrumbs	Deceive attackers
Compliance	audit, reports, compliance, policies, rules	Maintain governance
Integration	webhooks, siem, soar, api_keys, mcp	Connect external systems
Administration	users, roles, settings, backup, updates	System management

Plugin System

Extend Seraph without modifying core code:

```
# Example: Custom detection plugin
from seraph.plugins import DetectionPlugin, register_plugin

@register_plugin
class CustomRansomwareDetector(DetectionPlugin):
    """Detect custom ransomware variant targeting our industry."""

    name = "industry_ransomware_detector"
    version = "1.0.0"
    author = "Security Team"

    # Hook into file monitoring events
    triggers = ["file_created", "file_modified", "file_renamed"]

    def analyze(self, event: FileEvent) -> Optional[Detection]:
        # Custom detection logic
        if self._matches_industry_ransomware_pattern(event):
            return Detection(
                severity="critical",
                confidence=0.95,
                mitre_technique="T1486",
                details={"pattern": "industry-specific", "variant": "custom"}
            )
        return None

    def _matches_industry_ransomware_pattern(self, event: FileEvent) -> bool:
        # Industry-specific indicators
        patterns = [".industry_encrypted", ".sector_locked"]
        return any(event.path.endswith(p) for p in patterns)
```

Configuration Flexibility

Every component can be configured via environment variables, YAML files, or API:

```
# seraph-config.yaml
server:
  host: "0.0.0.0"
  port: 8000
  workers: 4

detection:
  ml_models:
    isolation_forest:
      enabled: true
      contamination: 0.1
    neural_network:
      enabled: true
      layers: [64, 32, 16]

  thresholds:
    critical: 90
    high: 70
    medium: 50

response:
  auto_quarantine:
    enabled: true
    min_score: 85
  auto_isolate:
    enabled: false # Require approval

integrations:
  siem:
    type: "splunk"
    endpoint: "https://splunk.company.com:8088"
    token: "${SPLUNK_HEC_TOKEN}"

  slack:
    enabled: true
    webhook: "${SLACK_WEBHOOK_URL}"
    channels:
      critical: "#soc-critical"
      high: "#soc-alerts"
```

How Seraph Defends Your Organization

This section explains the specific protection mechanisms Seraph provides against real-world threats.

Protection Against Ransomware

LAYER 1: PREVENTION

- ✓ Known ransomware hash blocking (100,000+ signatures)
- ✓ Suspicious file origin analysis (downloads, email attachments)
- ✓ Macro execution control (Office documents)
- ✓ Script interpreter monitoring (PowerShell, VBScript, WScript)

LAYER 2: DETECTION

- ✓ Entropy analysis: Detect encrypted file creation
- ✓ Bulk file modification: >50 files/minute = alert
- ✓ Shadow copy deletion: Immediate critical alert
- ✓ Ransom note detection: Pattern matching on file content
- ✓ Extension changes: Mass renaming to .encrypted, .locked, etc.

LAYER 3: RESPONSE (AUTOMATIC)

- ✓ Instant process termination: Kill ransomware within 50ms
- ✓ Process tree kill: Stop child processes and persistence
- ✓ File rollback: Restore from shadow copies if available
- ✓ Network isolation: Prevent lateral spread
- ✓ Fleet-wide hash block: Protect all endpoints immediately

Ransomware Kill Chain Interruption:

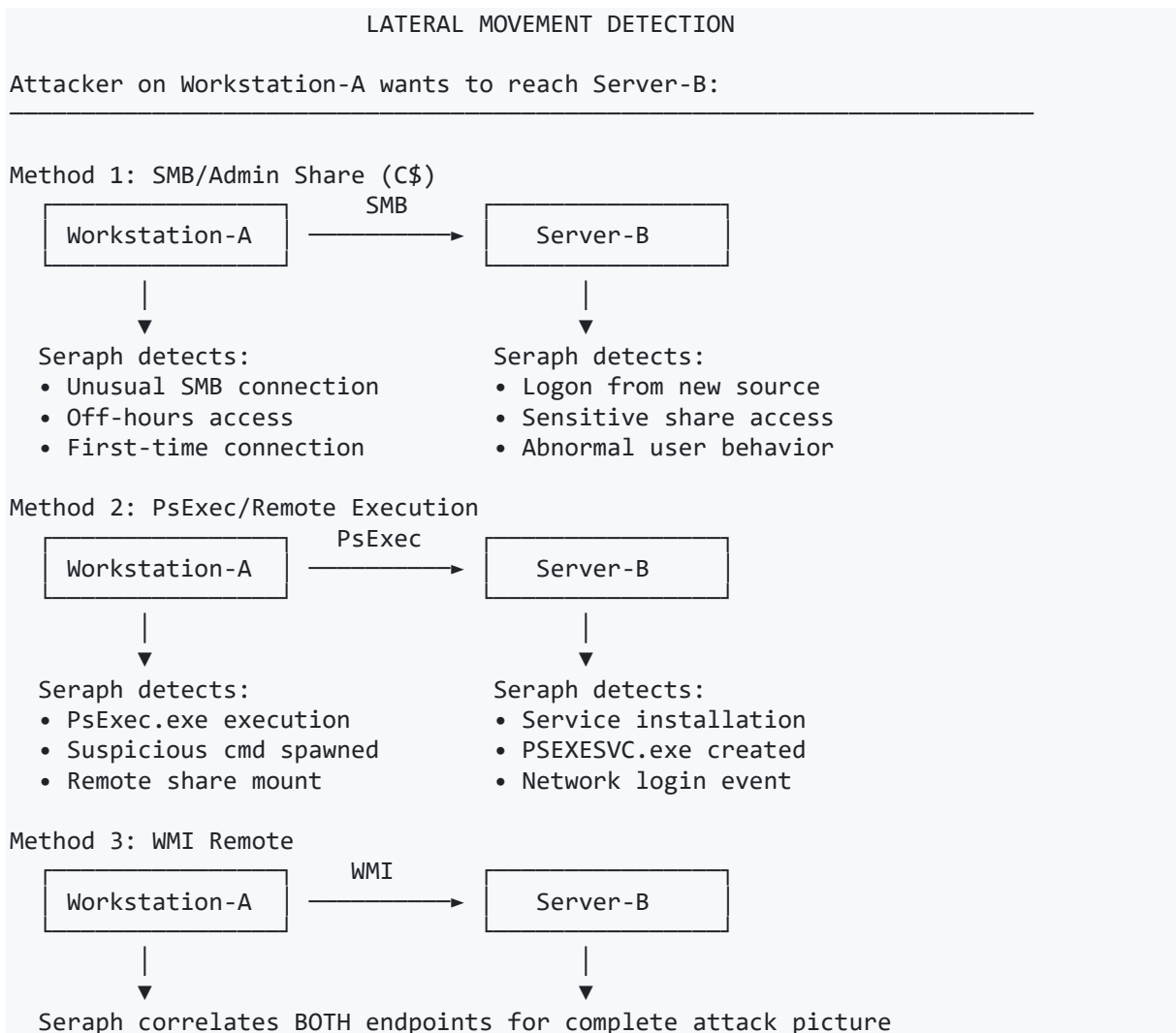


Protection Against Credential Theft

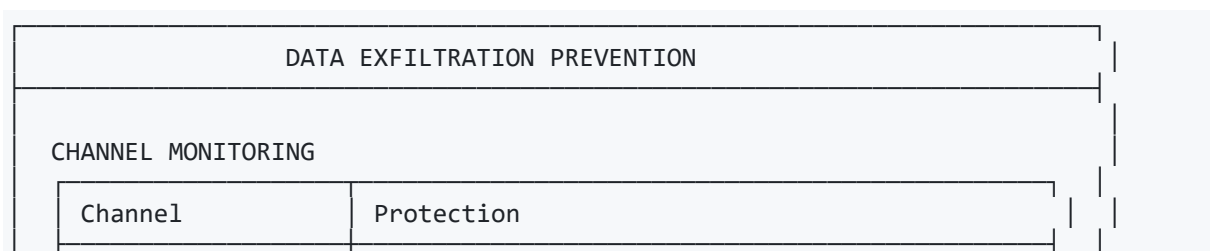
Attack Vector	Seraph Protection
Mimikatz/LSASS Dump	Memory protection hooks detecting LSASS access, process injection into LSASS, credential dumping patterns
Keyloggers	Keyboard hook detection, suspicious input capture patterns, clipboard monitoring for sensitive data
Pass-the-Hash	NTLM relay detection, unusual authentication patterns, impossible travel alerts

Attack Vector	Seraph Protection
Phishing	Browser isolation for suspicious links, URL reputation scoring, credential form detection
Credential Stuffing	Login velocity monitoring, failed auth correlation, honey credentials for detection

Protection Against Lateral Movement



Protection Against Data Exfiltration



HTTP/HTTPS DNS Email Cloud Storage USB Encrypted	DLP pattern matching, unusual upload sizes Tunneling detection, high-entropy subdomains Attachment analysis, recipient reputation Dropbox/GDrive/OneDrive policy enforcement Device control, file copy logging Cert pinning detection, TLS inspection points
VOLUME ANALYSIS	
• Baseline: Normal upload patterns per user/endpoint • Alert: >2x standard deviation from baseline • Block: >10GB outbound without approval • Investigate: Off-hours large transfers	
CONTENT AWARENESS	
• Credit card patterns (PCI) • Social security numbers (PII) • Source code patterns (.git, specific file extensions) • Custom patterns (company-specific data)	

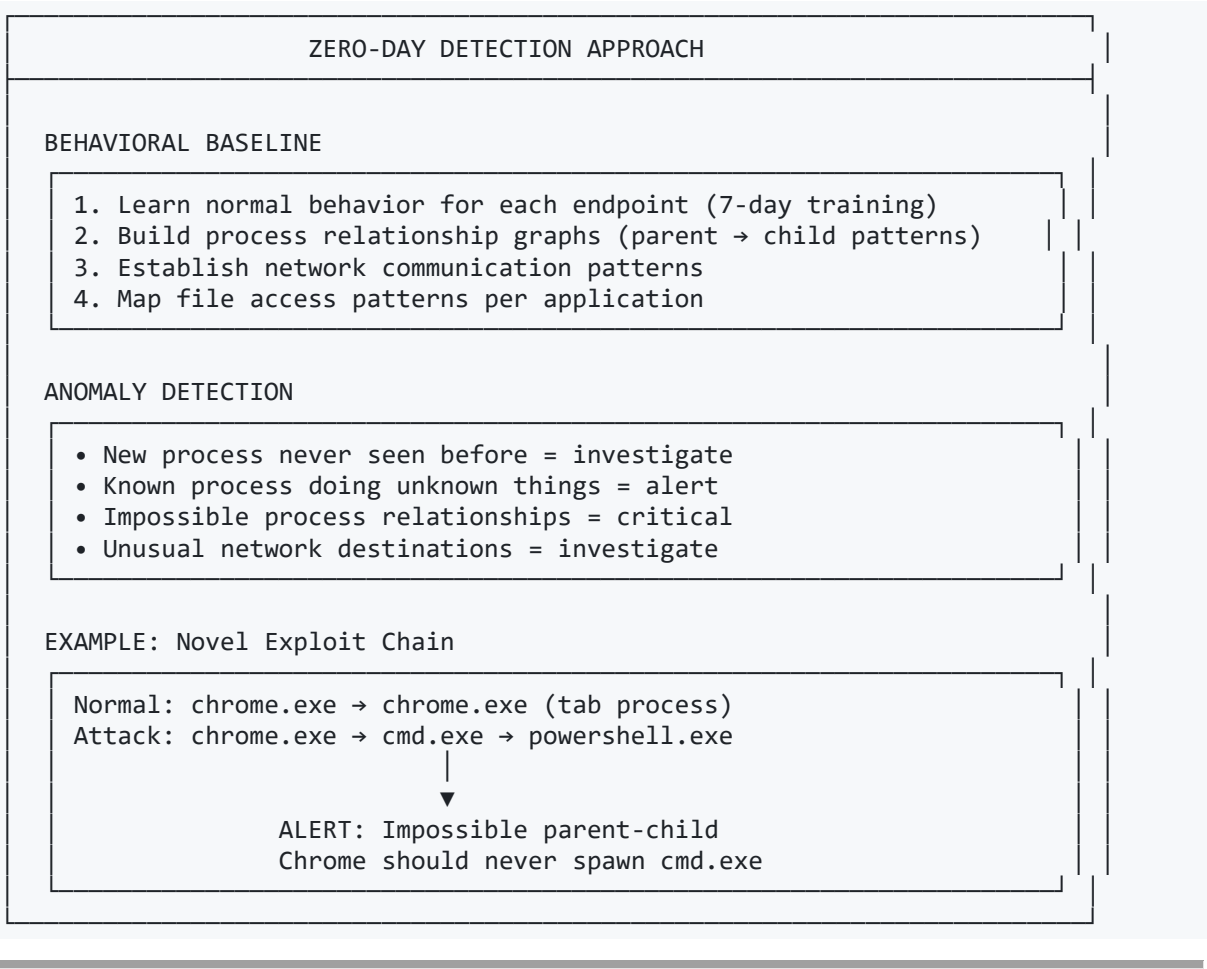
Protection Against Fileless Malware

Living-off-the-land attacks that use legitimate tools are increasingly common. Seraph detects these through:

Technique	Detection Method
PowerShell Abuse	Encoded command detection, suspicious cmdlets (IEX, Invoke-Expression), AMSI integration
WMI Persistence	Event subscription monitoring, suspicious WMI consumers, unusual namespaces
Registry Run Keys	Real-time registry monitoring, new Run/RunOnce entries, encoded payloads
Scheduled Tasks	Task creation monitoring, suspicious task paths, network-triggered tasks
DLL Hijacking	DLL load order monitoring, unsigned DLLs in system paths, phantom DLLs
Process Hollowing	Memory pattern analysis, section unmapping detection, suspicious thread creation

Zero-Day Protection

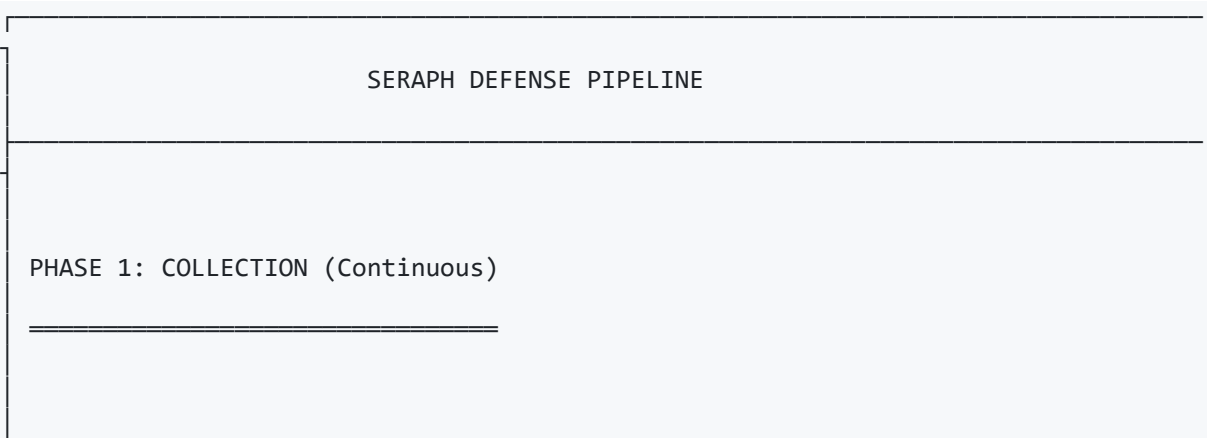
Even without signatures, Seraph can detect novel threats:

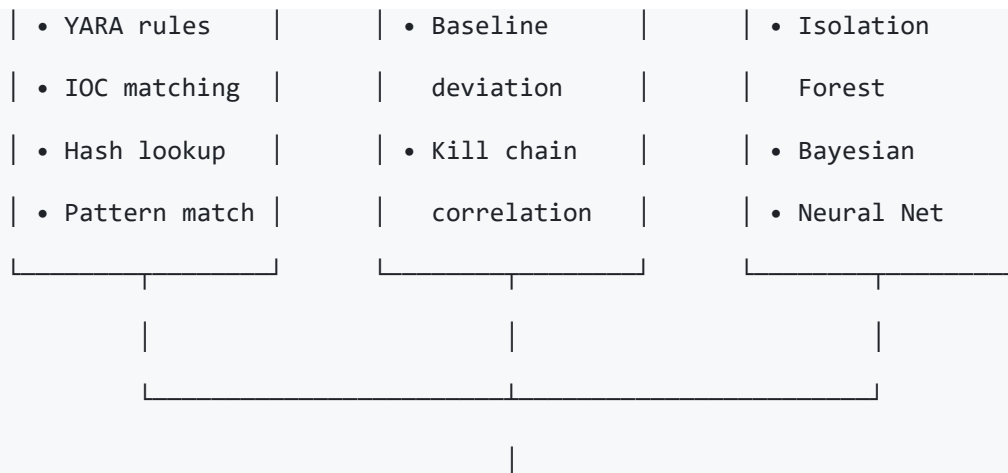


Complete Defense Pipeline

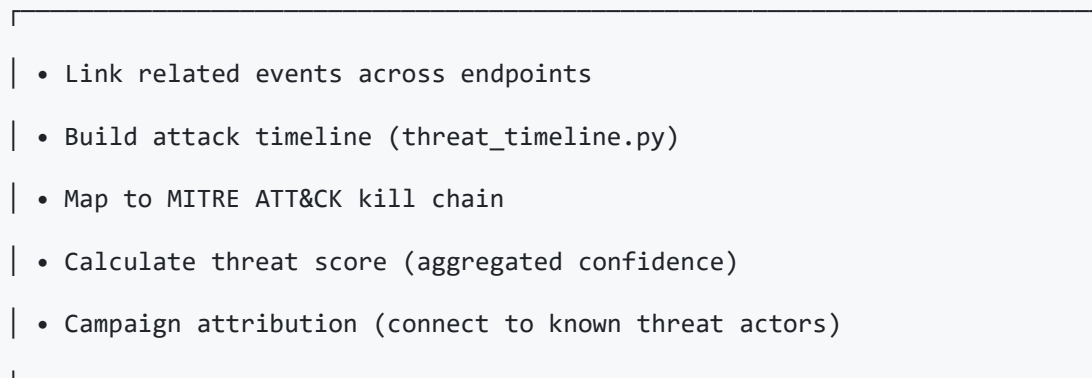
This section traces a threat from initial detection through complete remediation.

End-to-End Threat Lifecycle

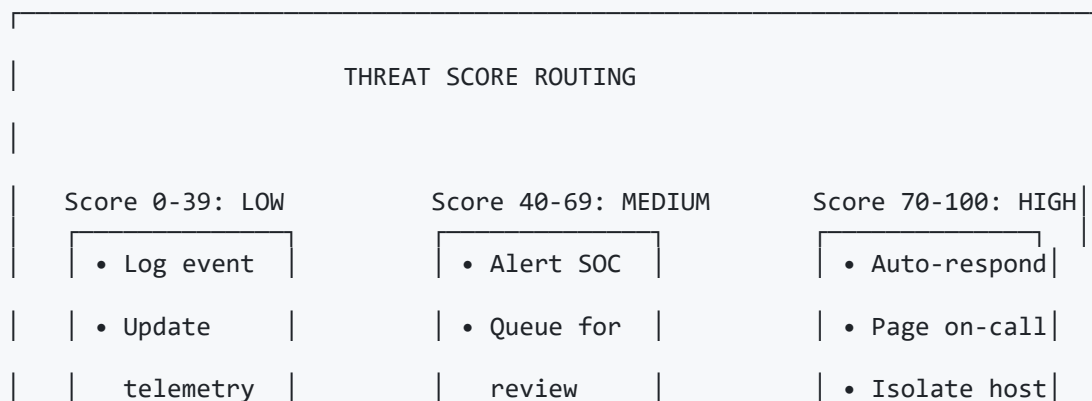




PHASE 4: CORRELATION



PHASE 5: TRIAGE



• Baseline	• Sandbox if	• Trigger
learning	file-based	playbook

|



PHASE 6: RESPONSE (SOAR)



PLAYBOOK EXECUTION	
Trigger: Critical threat detected on WORKSTATION-15	
Step 1: Isolate endpoint	✓ Network isolated in 2.3s
Step 2: Memory snapshot	✓ Forensic capture complete
Step 3: Kill malicious process	✓ PID 4532 terminated
Step 4: Quarantine files	✓ 3 files quarantined
Step 5: Hash block (fleet-wide)	✓ Hash blocked on 1,247 endpoints
Step 6: Notify SOC	✓ Slack + PagerDuty + Email sent
Step 7: Create incident	✓ INC-2024-0312 created
Step 8: Generate IOC report	✓ Report attached to incident
Playbook completed in 8.7 seconds (automated)	

|



PHASE 7: INVESTIGATION



--

| Analyst reviews incident with full context: |

|

- | • Timeline view: Minute-by-minute attack progression |
- | • Process tree: Complete execution chain with CLI arguments |
- | • Network map: All connections made by malicious process |
- | • File activity: Every file touched, created, modified |
- | • Memory analysis: Extracted strings, injected code, artifacts |
- | • MITRE mapping: Kill chain stage and technique breakdown |
- | • Similar incidents: Historical matches in environment |

|



PHASE 8: REMEDIATION

- | • Clean infected systems (restore from backup or reimage) |
- | • Reset compromised credentials |
- | • Patch exploited vulnerabilities |
- | • Update detection rules based on learnings |
- | • Close incident with full documentation |

|



PHASE 9: INTELLIGENCE FEEDBACK

- | • IOCs shared with threat intel platform |
- | • Detection rules updated with new signatures |
- | • ML models retrained with confirmed malicious samples |

	• Playbook improvements based on response effectiveness	
	• Briefing generated for security leadership	

Data Flow Metrics

Stage	Throughput	Latency	Retention
Collection	50,000 events/sec	Real-time	90 days raw
Normalization	50,000 events/sec	<10ms	N/A (streaming)
Detection	10,000 events/sec	<100ms	Detections: 1 year
Correlation	1,000 incidents/min	<500ms	Incidents: 2 years
Response	100 playbooks/min	Variable	Actions: 5 years

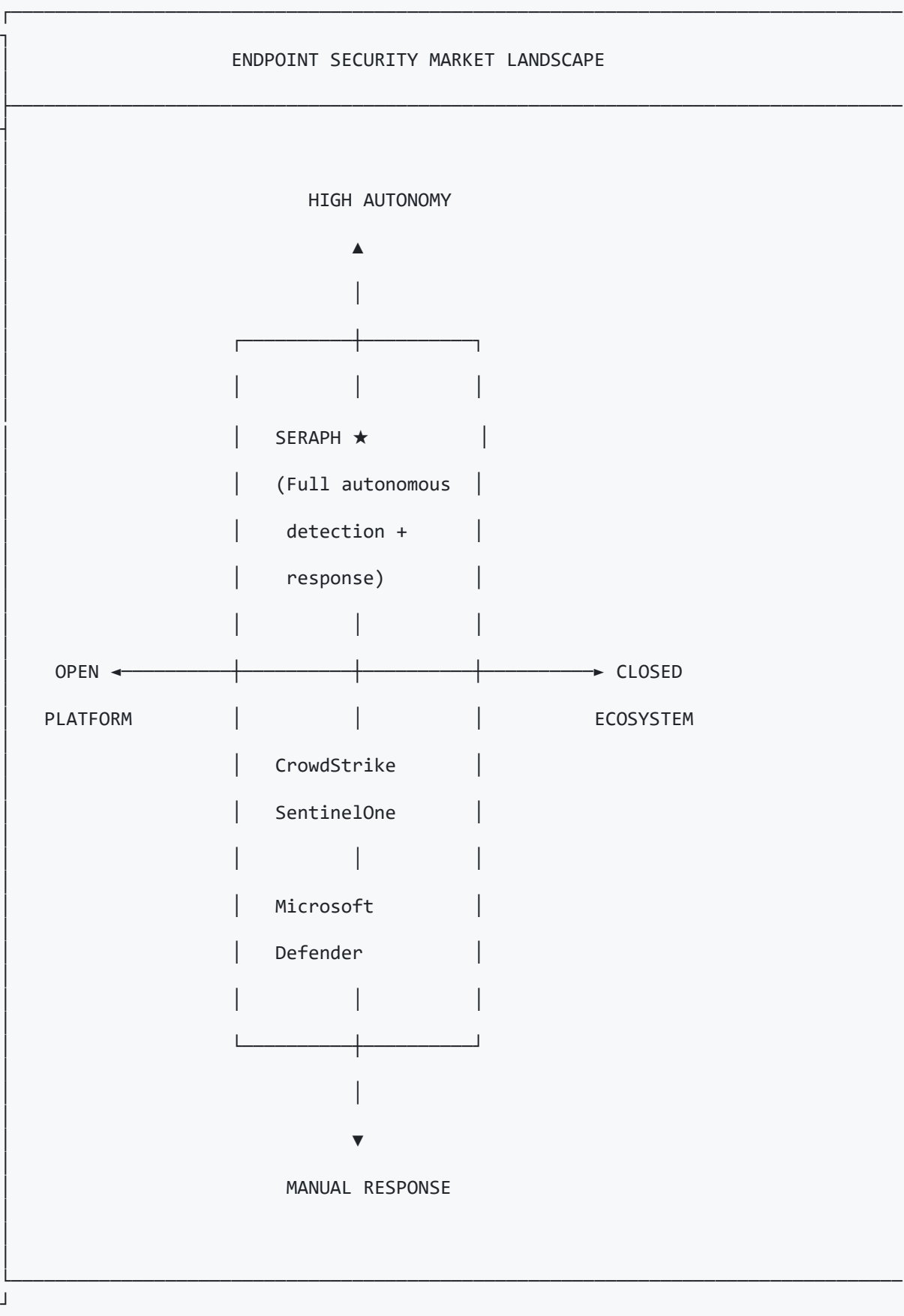
Critical Path Example: Ransomware Attack

TIME	EVENT	SERAPH ACTION
00:00.000	User opens email attachment	Sandbox analysis triggered
00:00.050	Macro executes PowerShell	Script monitoring alert
00:00.100	Encoded command detected	AMSI bypass attempt flagged
00:00.150	C2 beacon attempt	DNS anomaly detected
00:00.200	Privilege escalation attempt	UAC bypass detected
00:00.250	Shadow copy deletion	CRITICAL: Ransomware signature
00:00.300	— AUTOMATED RESPONSE BEGINS —	
00:00.350	Malicious process terminated	Kill process + tree
00:00.400	Endpoint isolated from network	Network isolation applied
00:00.450	Hash blocked fleet-wide	1,247 endpoints protected
00:00.500	SOC alert triggered	PagerDuty + Slack + Email
00:01.000	Memory snapshot captured	Forensics preserved
00:05.000	Incident created with full timeline	INC-2024-0312
TOTAL TIME TO CONTAINMENT: 400 milliseconds		

Competitive Differentiation

Seraph occupies a unique position in the endpoint security market. While established players like CrowdStrike, SentinelOne, and Microsoft Defender offer robust protection, Seraph's architecture addresses critical gaps that enterprises increasingly face in the age of AI-driven attacks.

Market Positioning



Feature Comparison Matrix

Capability	Seraph	CrowdStrike Falcon	Sentinel One	Microsoft Defender for Endpoint
Deployment Model	Self-hosted + Cloud	Cloud-only	Hybrid	Cloud (Intune)
AI/ML Detection	☒ On-device + server	☒ Cloud-based	☒ On-device	☒ Cloud-based
Zero-Dependency Agent	☒ Pure Python	☒ Requires runtime	☒ Requires runtime	☒ Windows components
Active Deception	☒ Full suite (Pebbles/Mystique/Stonewall)	☒ None	☒ Basic honeypots	☒ None
MCP/AI Agent Integration	☒ Native	☒ None	☒ None	☒ Copilot (limited)
Open API	☒ Full OpenAPI spec	☒ Partial	☒ Partial	☒ Graph API
Source Available	☒ Full source	☒ Proprietary	☒ Proprietary	☒ Proprietary
SOAR Built-in	☒ Native playbooks	☒ Extra cost	☒ Extra cost	☒ Sentinel required
Cross-Platform Agent	☒ Single codebase	☒ Separate agents	☒ Separate agents	☒ Windows-focused

Capability	Seraph	CrowdStrike Falcon	Sentinel One	Microsoft Defender for Endpoint
Swarm Auto-Deploy	☒ Yes	☒ Manual	☒ Manual	☒ Intune required
Attack Timeline	☒ MITRE-mapped	☒ Yes	☒ Yes	☒ Basic
Ransomware Rollback	☒ Shadow copy restore	☒ Yes	☒ Yes	☒ Windows only
Browser Isolation	☒ Built-in	☒ Add-on	☒ Add-on	☒ None
Offline Detection	☒ Full capability	☒ Reduced	☒ Full	☒ Reduced
AATL (AI Agent Threat Layer)	☒ Native cognition engine	☒ None	☒ None	☒ None
Cognition Engine (CCE)	☒ CLI behavior analysis	☒ None	☒ None	☒ None
Human vs Machine Scoring	☒ Real-time behavioral	☒ None	☒ None	☒ None
Intent Accumulation	☒ 8 threat categories	☒ None	☒ None	☒ None
AI-Specific Response	☒ 6 adaptive strategies	☒ Block only	☒ Block only	☒ Block only
AATR (AI Threat Registry)	☒ Defensive threat catalog	☒ None	☒ None	☒ None

Capability	Seraph	CrowdStrike Falcon	Sentinel One	Microsoft Defender for Endpoint
Post-Quantum Cryptography	☑ NIST PQC (Kyber/Dilithium/SPHINCS+)	✗ None	✗ None	✗ None
Hybrid Encryption	☑ PQC + Classical	✗ None	✗ None	✗ None
Policy Governance (PDP/PEP)	☑ Full XACML-style	⚠ Basic RBAC	⚠ Basic RBAC	⚠ Azure AD
Two-Person Integrity	☑ Built-in approvals	✗ None	✗ None	✗ None
Trust State Machine	☑ 4 dynamic states	✗ Binary trust	✗ Binary trust	✗ Binary trust
Tool Gateway (PEP)	☑ Allowlisted CLI execution	✗ None	✗ None	✗ None
Capability Token Broker	☑ Scoped secrets vault	✗ None	✗ None	⚠ Azure Key Vault
Tamper-Evident Telemetry	☑ Hash-chained, court-admissible	⚠ Logs only	⚠ Logs only	⚠ Logs only
Identity Protection	☑ AD/Kerberos/LDAP attack detection	☑ Falcon Identity	⚠ Singularity Identity	☑ Defender for Identity
Kerberoasting Detection	☑ Encryption downgrade analysis	☑ Yes	⚠ Basic	☑ Yes

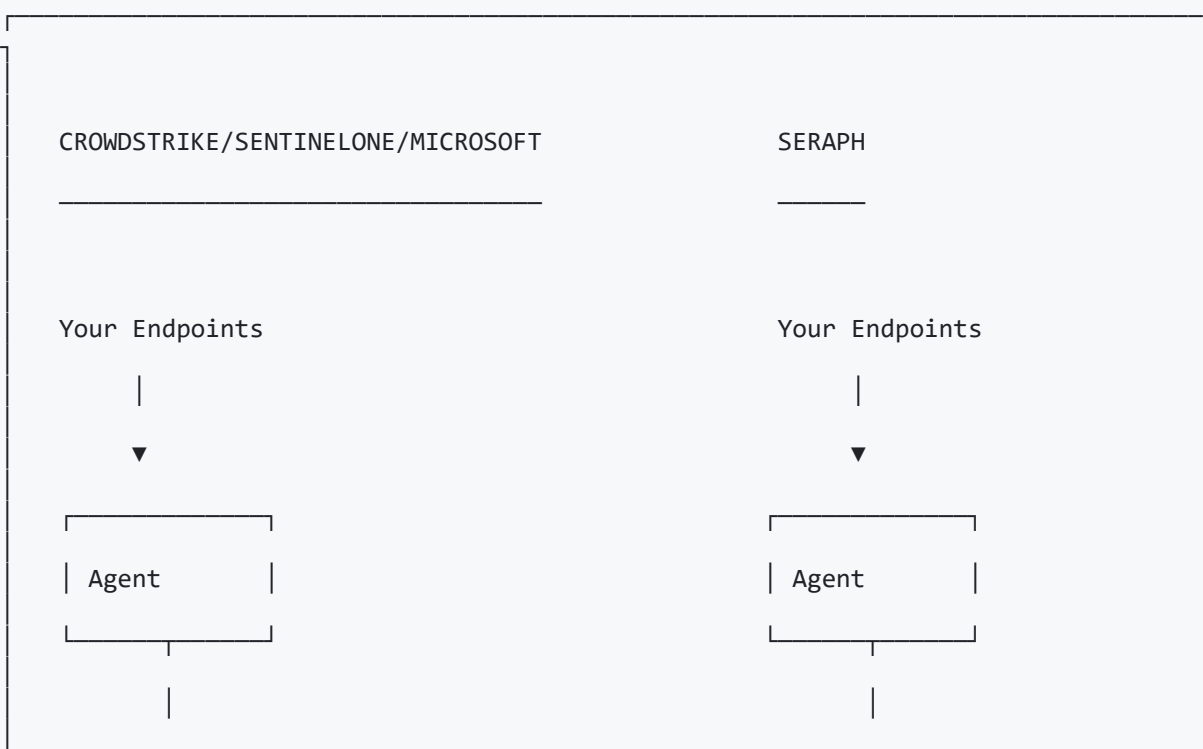
Capability	Seraph	CrowdStrike Falcon	Sentinel One	Microsoft Defender for Endpoint
Golden/Silver Ticket Detection	☑ Ticket anomaly detection	☑ Yes	⚠ Basic	☑ Yes
DCSync/DCShadow Detection	☑ Replication monitoring	☑ Yes	⚠ Basic	☑ Yes
Container Security	☑ Native (Trivy + Falco)	⚠ Add-on	⚠ Add-on	⚠ Basic
Image Vulnerability Scanning	☑ Automated Trivy	⚠ Cloud Workload	⚠ Cloud Workload	⚠ Defender for Cloud
K8s Runtime Security	☑ Falco integration	⚠ Add-on	⚠ Add-on	⚠ AKS only
Image Signing Verification	☑ Cosign/Notary support	✗ None	✗ None	✗ None
CIS Benchmark Compliance	☑ Docker + K8s	⚠ Basic	⚠ Basic	⚠ Basic
Vector Memory Database	☑ Semantic search + embeddings	✗ None	✗ None	✗ None
Case-Based Reasoning	☑ Incident learning	✗ None	✗ None	✗ None
Automated Threat Hunting	☑ 100+ MITRE rules	☑ OverWatch (\$\$)	⚠ Basic	⚠ Basic
Local AI Reasoning	☑ Ollama/llama.cpp ready	✗ None	✗ None	✗ None

Capability	Seraph	CrowdStrike Falcon	Sentinel One	Microsoft Defender for Endpoint
Multi-Tenant Architecture	☒ Full tenant isolation	☒ Yes	☒ Yes	☒ Yes
SIEM Integration	☒ Splunk/ELK/QRadar	☒ Yes	☒ Yes	☒ Yes
Cuckoo Sandbox Integration	☒ Native behavioral analysis	☒ None	☒ None	☒ None
Custom Detection Plugins	☒ Python SDK	☒ None	☒ Limited	☒ None

Key Differentiators

1. Self-Hosted + Full Source Access

Unlike SaaS-only competitors, Seraph can run entirely in your infrastructure:





Use Cases:

- Government/defense contractors requiring air-gapped deployments
- Healthcare organizations with strict HIPAA requirements
- Financial institutions with data residency mandates
- Organizations wanting to avoid vendor dependency

2. Native AI Agent Integration (MCP)

Seraph is the only endpoint security platform with native Model Context Protocol support:





3. Advanced Deception Technology

No competitor offers the depth of deception capabilities:

Deception Feature	Seraph	CrowdStrike	SentinelOne	Microsoft
Honey credentials	✓	✗	⚠ Limited	✗
Honey files/documents	✓	✗	⚠ Limited	✗

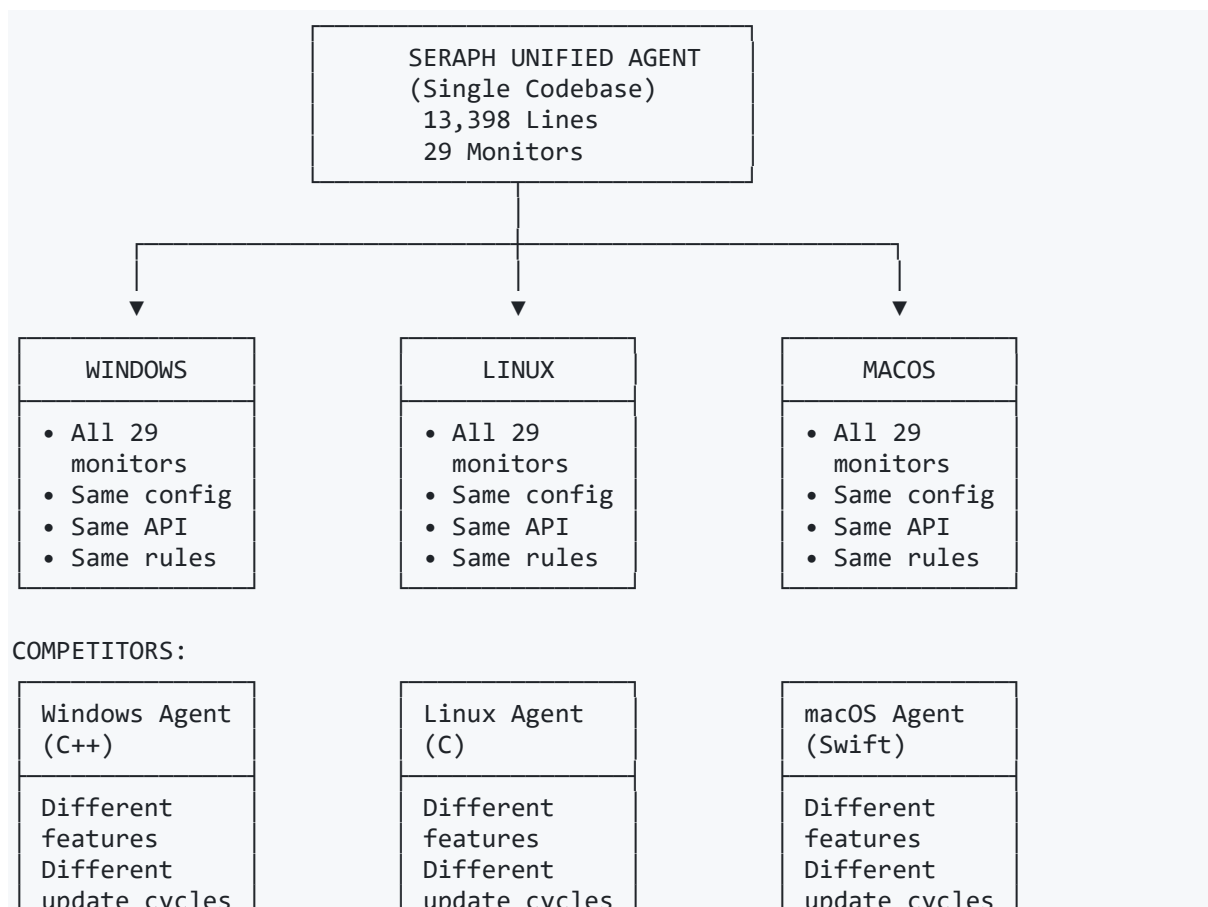
Deception Feature	Seraph	CrowdStrike	SentinelOne	Microsoft
Fake API keys	☑	✗	✗	✗
Campaign tracking	☑	✗	✗	✗
Adaptive deception	☑	✗	✗	✗
Progressive escalation	☑	✗	✗	✗
Attack path visualization	☑	✗	✗	✗

Why This Matters:

Traditional EDR is reactive—it waits for attacks to happen, then responds. Seraph's deception layer is proactive—it detects attackers during reconnaissance, before they reach their objectives, and wastes their time with fake targets.

4. True Cross-Platform Unified Agent

Single Python codebase runs everywhere:



5. Cost Model Advantage

TOTAL COST OF OWNERSHIP (1,000 ENDPOINTS)

CROWDSTRIKE FALCON ENTERPRISE

- Per-endpoint license: $\$180/\text{year} \times 1,000 = \$180,000/\text{year}$
- Cloud storage overages: $\sim \$12,000/\text{year}$
- Professional services: $\sim \$50,000$ (implementation)
- Year 1 Total: $\sim \$242,000$
- Year 3 Total: $\sim \$626,000$

SENTINELONE COMPLETE

- Per-endpoint license: $\$150/\text{year} \times 1,000 = \$150,000/\text{year}$
- Ranger add-on: $\sim \$30,000/\text{year}$
- Professional services: $\sim \$40,000$ (implementation)
- Year 1 Total: $\sim \$220,000$
- Year 3 Total: $\sim \$580,000$

SERAPH (SELF-HOSTED)

- License: Open source / Enterprise support optional
- Infrastructure: $\sim \$2,000/\text{month}$ (3 servers) = $\$24,000/\text{year}$
- Internal engineering: $\sim \$30,000/\text{year}$ (part-time)
- Year 1 Total: $\sim \$54,000$
- Year 3 Total: $\sim \$162,000$

SAVINGS WITH SERAPH: 70-75% reduction in TCO

When to Choose Seraph

Scenario	Recommendation
Data sovereignty requirements	☒ Choose Seraph (self-hosted)
AI/LLM integration needs	☒ Choose Seraph (native MCP)
Budget constraints	☒ Choose Seraph (no per-seat)
Want source code access	☒ Choose Seraph (open source)
Need advanced deception	☒ Choose Seraph (unique capability)
Enterprise support SLA	Consider CrowdStrike/SentinelOne
Deep Microsoft integration	Consider Microsoft Defender
Threat hunting services	Consider CrowdStrike OverWatch

Technology Advantages

Technical Aspect	Seraph Advantage
No runtime dependencies	Agent works even if Python isn't installed (bundled)
Offline-first design	Full detection capability without cloud connectivity
API-first architecture	Every feature accessible via REST API
Webhook integrations	Native support for any HTTP-capable system
Custom playbooks	YAML-based, version-controlled response automation
Plugin system	Extend detection without modifying core code

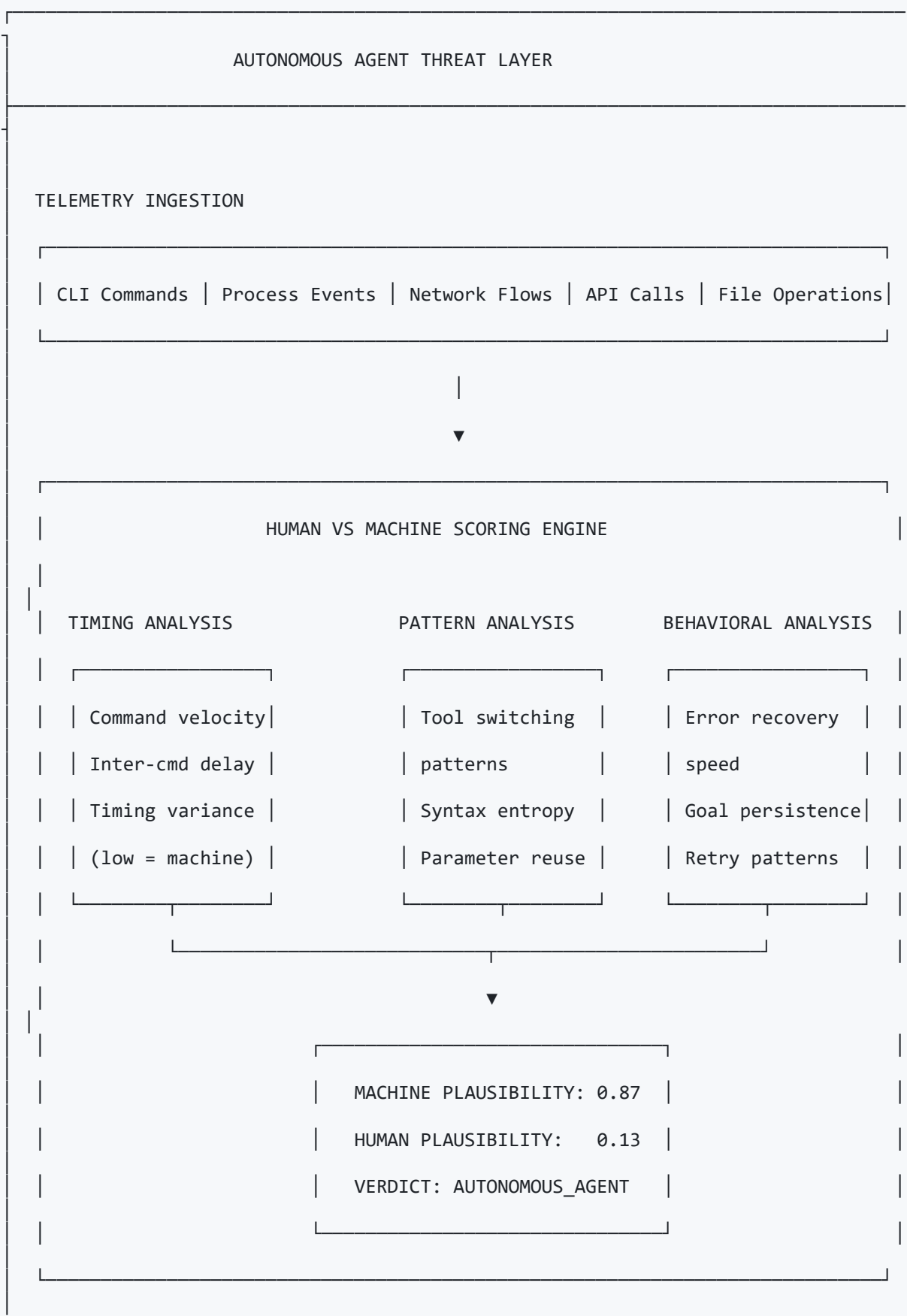
Autonomous Agent Threat Layer (AATL)

Seraph is the **only endpoint security platform** purpose-built to detect and respond to **AI-driven attacks**. The Autonomous Agent Threat Layer (AATL) treats AI agents as a first-class threat category with specialized detection, scoring, and response strategies.

Why AATL Matters

THE AI ATTACK LANDSCAPE (2025+)	
TRADITIONAL THREATS	AI-DRIVEN THREATS
• Human-paced attacks • Manual reconnaissance • Script kiddie toolkits • Predictable patterns • Single attack vector • Hours/days to pivot	• Sub-second decision making • Automated vulnerability scanning • LLM-generated exploits • Adaptive, self-modifying behavior • Multi-vector simultaneous attacks • Seconds to pivot
DETECTION CHALLENGE: Traditional EDR watches for known malware hashes and suspicious behaviors. AI agents use legitimate tools (curl, ssh, python) to achieve malicious goals—they evade signature-based detection entirely. SERAPH SOLUTION: AATL Instead of asking "Is this file malicious?", AATL asks: "Is this behavior human or machine? What is the intent?"	

AATL Architecture





INTENT ACCUMULATION

Recent Commands:	Intent Classification:
[06:32:01] whoami	_____
[06:32:01] id	reconnaissance: 45%
[06:32:02] cat /etc/passwd	credential_access: 35%
[06:32:02] find / -name "*.pem"	lateral_movement: 12%
[06:32:03] cat ~/.ssh/id_rsa	exfil_prep: 8%

PRIMARY INTENT: reconnaissance → credential_access

GOAL CONVERGENCE: 0.91 (highly focused attack)



LIFECYCLE STAGE DETECTION

RECON → ACCESS → EXECUTION → PERSISTENCE → PRIV_ESC → ...

▲

└─ CURRENT STAGE (with 78% confidence)

12-Stage Kill Chain: Reconnaissance → Initial Access → Execution → Persistence → Privilege Escalation → Defense Evasion → Credential Access → Discovery → Lateral Movement → Collection → Exfiltration → Impact



Actor Classification

AATL classifies threat actors into four categories:

Actor Type	Characteristics	Detection Signals
HUMAN	Variable timing, typos, breaks, context switching	>200ms inter-command delay, high variance, error correction
AUTOMATED_SCRIPT	Fixed timing, repetitive patterns	Exact timing intervals, no error handling
AI_ASSISTED	Human with AI help	Mixed timing, sophisticated commands, occasional pauses

Actor Type	Characteristics	Detection Signals
AUTONOMOUS_AGENT	Pure AI execution	Sub-100ms timing, low variance, adaptive retries, goal persistence

Trusted AI Whitelist

AATL includes a sophisticated whitelist for legitimate AI tools:

```
TRUSTED_AI_TOOLS = {
  # IDE AI Assistants
  "code", "vscode", "copilot-agent", "cursor", "jetbrains",

  # AI Coding Tools
  "claude", "ollama", "chatgpt", "aider", "codeium", "tabnine",

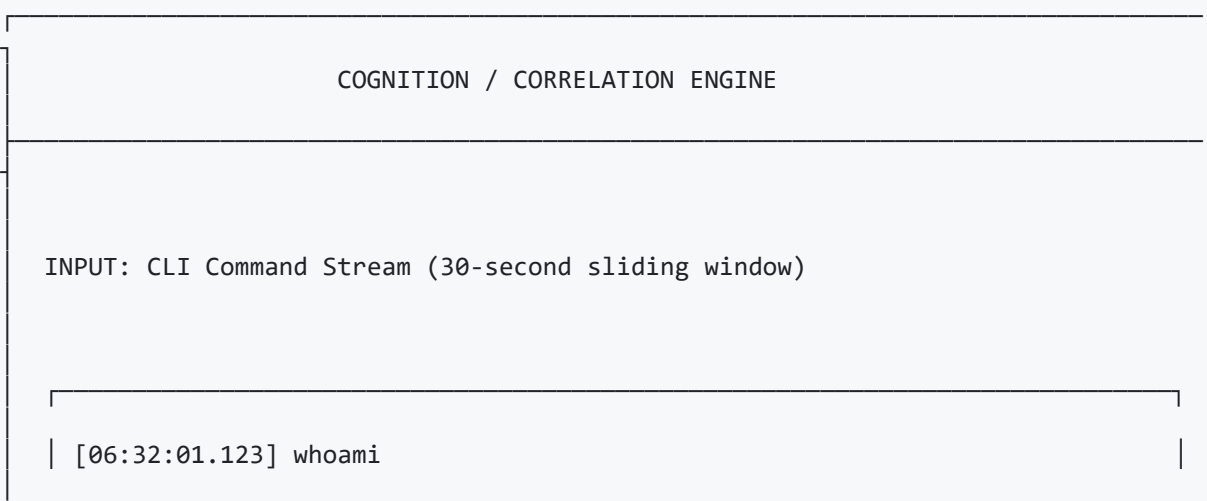
  # Our own infrastructure
  "mcp-server", "metatron-mcp", "seraph-defender", "unified-agent"
}
```

When commands originate from trusted sources, AATL skips threat analysis—ensuring legitimate development workflows aren't interrupted while maintaining vigilance against malicious AI.

Cognition / Correlation Engine (CCE)

The Cognition Engine works alongside AATL to analyze command streams and detect machine-paced autonomous behavior in real-time.

CCE Analysis Pipeline



```
| [06:32:01.287] id
| [06:32:01.451] hostname
| [06:32:01.612] cat /etc/passwd
| [06:32:01.789] find / -name "*.pem" 2>/dev/null
| ...
```

ANALYSIS MODULES

MACHINE LIKELIHOOD SCORING

- Average inter-command delay: 164ms (threshold: 200ms = suspicious)
- Timing variance (σ): 23ms (threshold: 50ms = machine-like)
- Command burst analysis: 5 commands in 0.67s = HIGH velocity
- RESULT: machine_likelihood = 0.89

BURSTINESS ANALYSIS

- Coefficient of variation in timing
- Burst detection (>3 commands in <500ms)
- RESULT: burstiness_score = 0.78

TOOL SWITCH LATENCY

- Time between different tool categories
- recon → creds: 276ms (humans: typically >2s)
- RESULT: tool_switch_latency_ms = 276

GOAL PERSISTENCE

- Retry patterns after errors
- Command variation on retry (adaptive = AI)
- RESULT: goal_persistence = 0.92

INTENT CLASSIFICATION

Patterns matched:

- "whoami", "id", "hostname" → recon
- "cat /etc/passwd" → credential_access
- "find -name *.pem" → credential_access
- RESULT: dominant_intents = ["recon", "credential_access"]

OUTPUT: cli.session_summary

```
{
  "event_type": "cli.session_summary",
  "machine_likelihood": 0.89,
  "burstiness_score": 0.78,
  "tool_switch_latency_ms": 276,
  "goal_persistence": 0.92,
  "dominant_intents": ["recon", "credential_access"],
  "command_count": 47,
  "decoy_touched": false,
  "verdict": "AUTONOMOUS_AGENT_SUSPECTED"
}
```

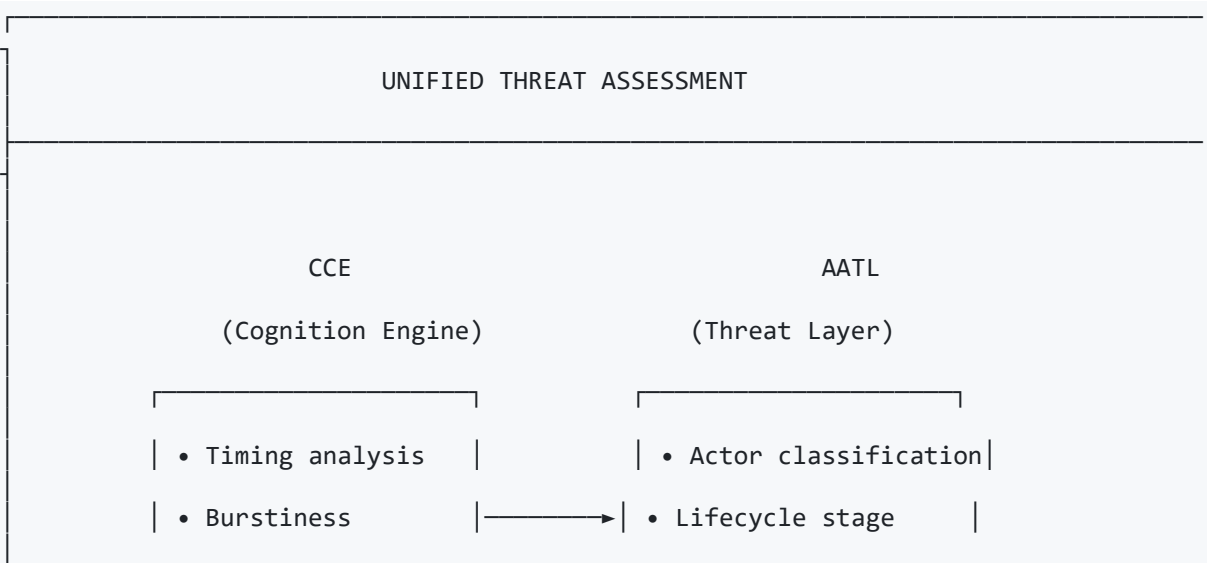


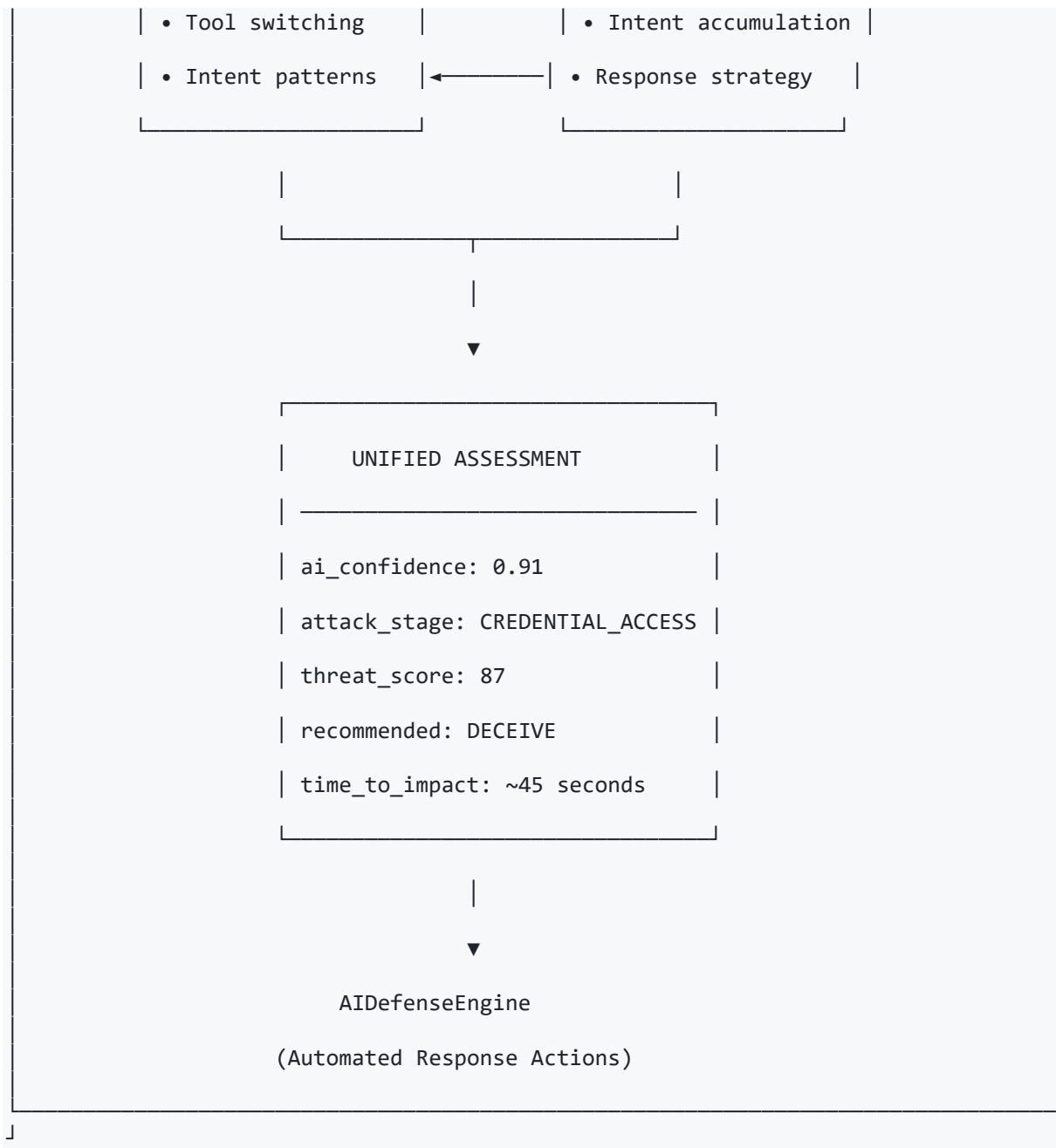
Intent Pattern Categories

The CCE classifies commands into 8 intent categories:

Intent	Example Patterns	Risk Level
recon	whoami, id, hostname, netstat, arp -a, net user	Low
credential_access	mimikatz, cat ~/.ssh/id_rsa, reg query SAM, hashdump	Critical
lateral_movement	psexec, ssh, wmiexec, Invoke-Command, net use \\	High
privilege_escalation	sudo, runas, getsystem, juicypotato, linpeas	High
persistence	schtasks /create, crontab, reg add...Run, systemctl enable	High
defense_evasion	Clear-EventLog, rm .bash_history, timestomp, AMSI bypass	High
exfil_prep	tar -czf, zip -r, base64, certutil -encode	Medium
data_staging	find -name *.doc, copy \\share\, robocopy	Medium

AATL ↔ CCE Integration





Post-Quantum Cryptography

Seraph implements **NIST-standardized post-quantum cryptographic algorithms** to protect against future quantum computing threats. This makes Seraph one of the first endpoint security platforms to offer quantum-resistant data protection.

Why Quantum Security Matters

THE QUANTUM THREAT

TODAY'S ENCRYPTION

AFTER QUANTUM COMPUTERS

RSA-2048: Secure

RSA-2048: Broken in hours

ECDSA: Secure

ECDSA: Broken in hours

AES-256: Secure

AES-256: Weakened (128-bit effective)

SHA-256: Secure

SHA-256: Weakened (Grover's algorithm)

"HARVEST NOW, DECRYPT LATER" ATTACK:

Adversaries are collecting encrypted data TODAY, waiting for quantum computers to decrypt it in 5-10 years. Sensitive data with long-term value (health records, financial data, state secrets) is at risk NOW.

SERAPH SOLUTION:

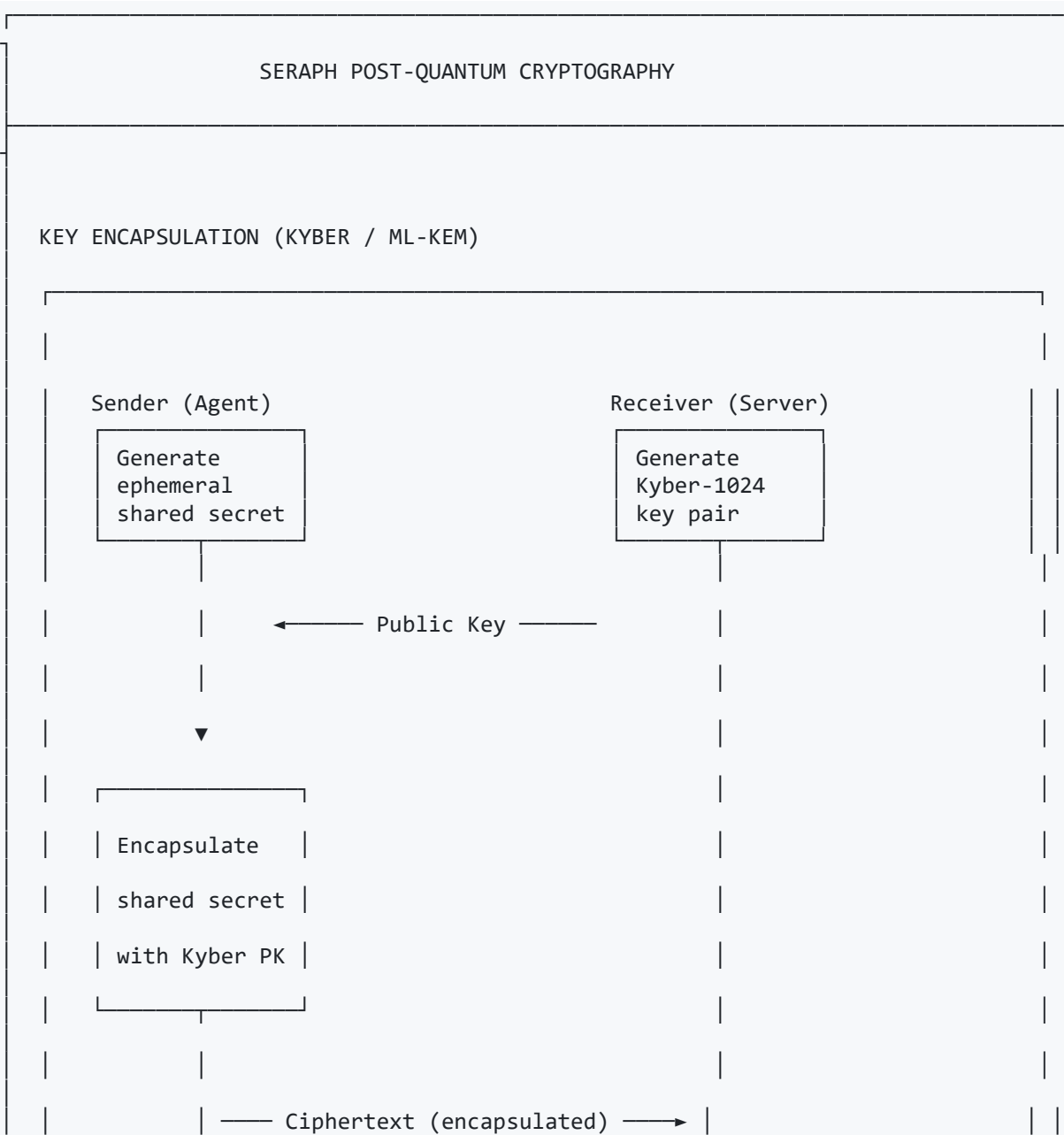
Implement NIST PQC standards TODAY to protect data that needs to remain confidential for years or decades.

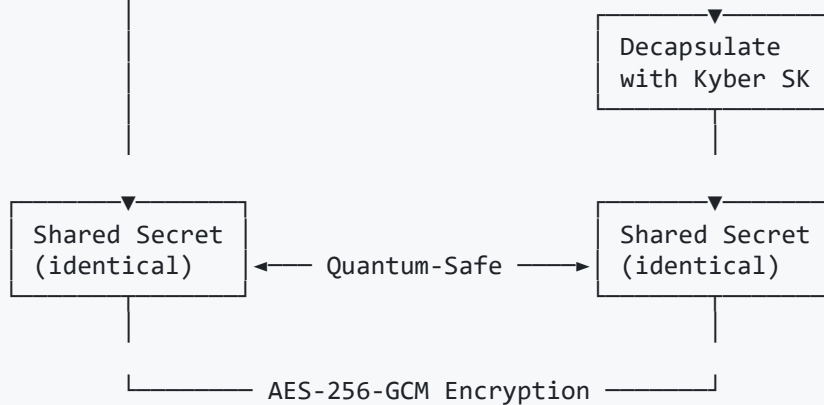
Supported Algorithms (NIST FIPS 203/204/205)

Algorithm	Type	Security Level	Use Case
CRYSTALS-Kyber (ML-KEM)	Key Encapsulation	128/192/256-bit	Secure key exchange, TLS

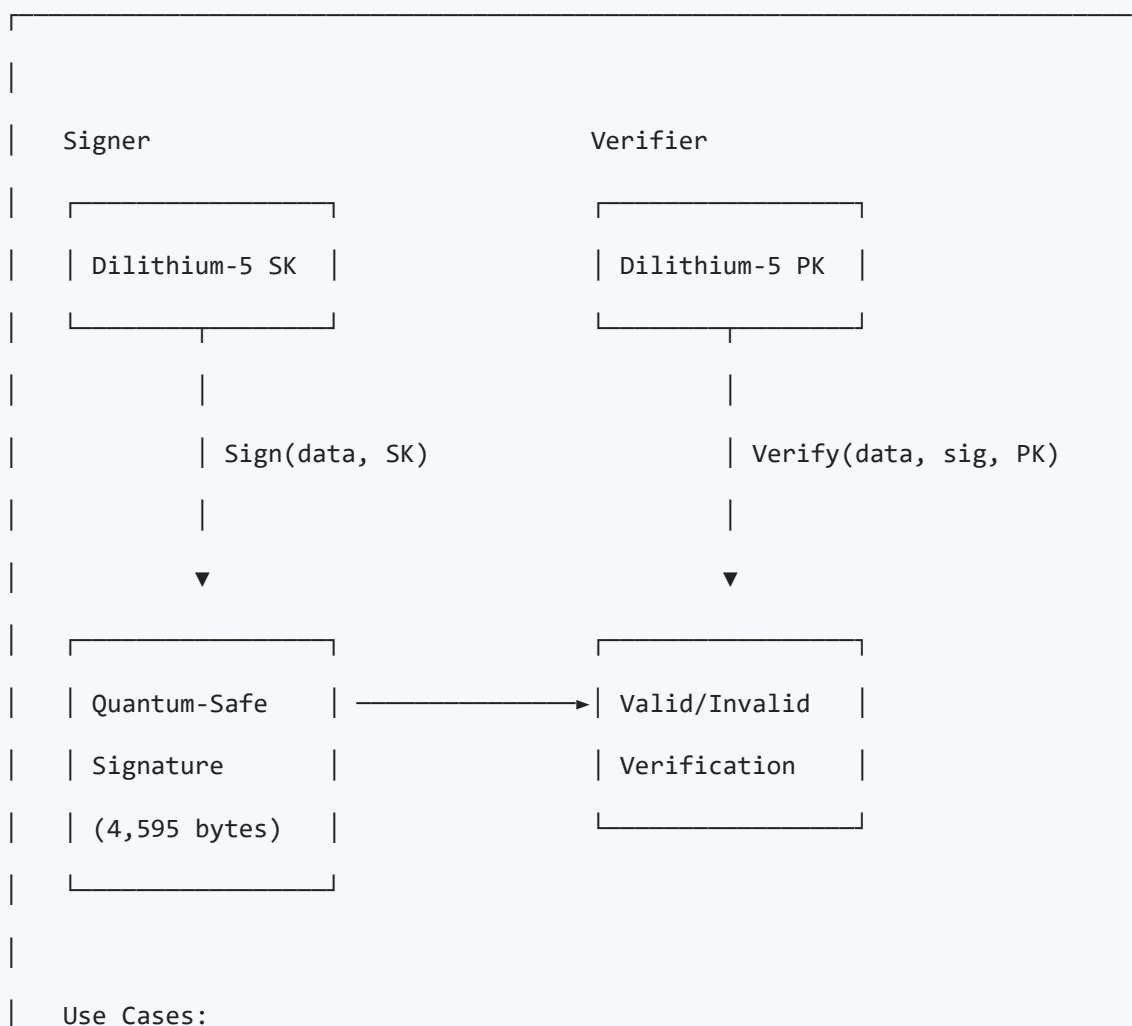
Algorithm	Type	Security Level	Use Case
CRYSTALS-Dilithium (ML-DSA)	Digital Signature	128/192/256-bit	Code signing, authentication
SPHINCS+ (SLH-DSA)	Hash-based Signature	128/256-bit	Maximum security, firmware
Hybrid Modes	KEM + Classical	Defense-in-depth	Kyber + X25519, Kyber + P-384

Quantum Security Architecture





DIGITAL SIGNATURES (DILITHIUM / ML-DSA)



- Agent binary signing
- Configuration integrity
- Audit log non-repudiation
- Playbook authentication

HYBRID ENCRYPTION



Defense-in-depth: Even if Kyber is broken, X25519 provides protection (assuming classical computers remain relevant)

MODES OF OPERATION

- SIMULATION: Pure Python implementation (always available)
- LIBOQS: Production mode using Open Quantum Safe library
- PQCRYPTO: Production mode using pqcrypto Python bindings

Key Management:

- Automatic key rotation (configurable intervals)
- Usage-based rotation (max operations per key)
- HSM integration patterns
- Shamir Secret Sharing for key escrow

Quantum Security Status API

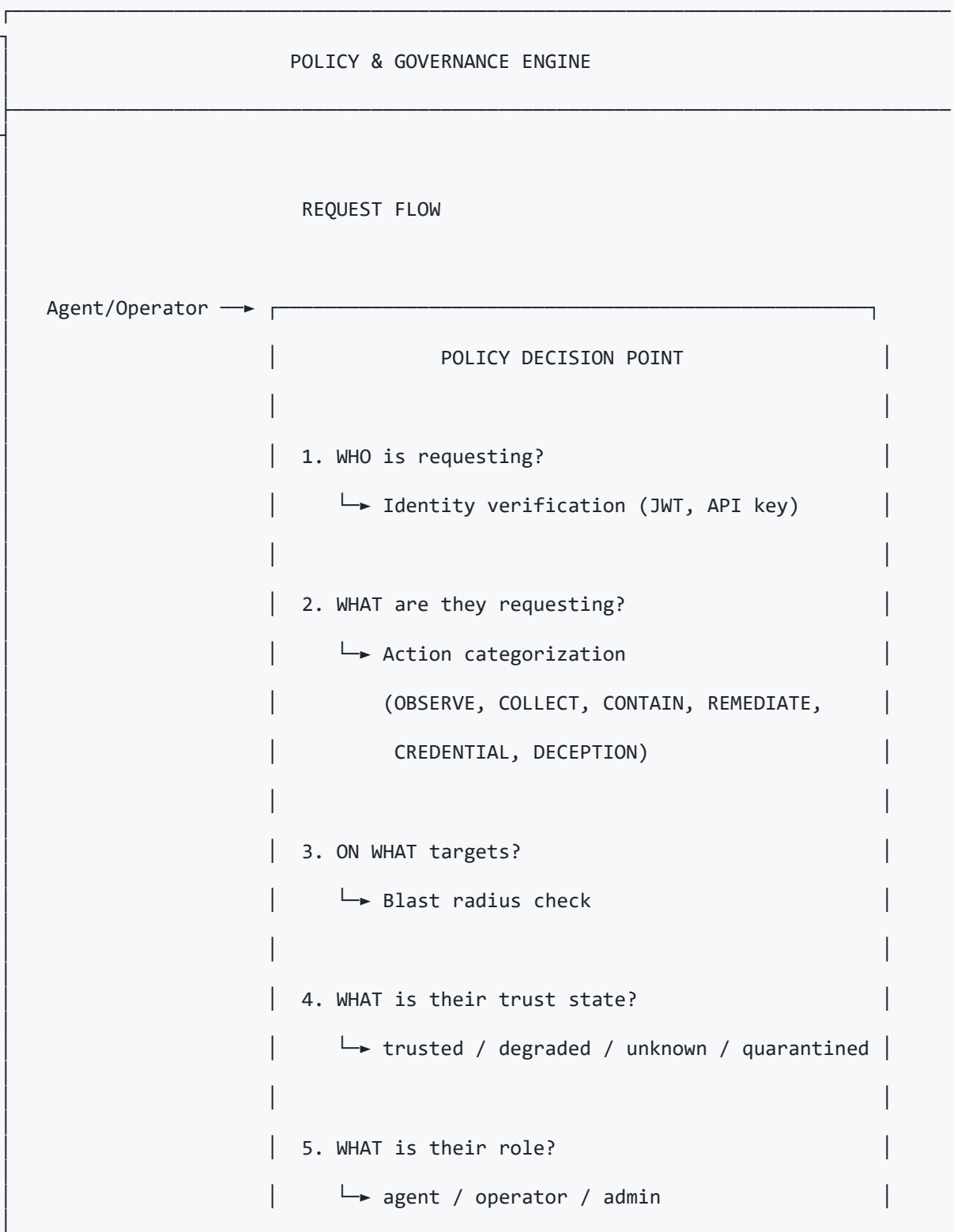
GET /api/quantum/status

```
{
  "mode": "liboqs",
  "algorithms": {
    "kem": ["KYBER-512", "KYBER-768", "KYBER-1024"],
    "signatures": ["DILITHIUM-2", "DILITHIUM-3", "DILITHIUM-5", "SPHINCS+-256"],
    "hash": "SHA3-256"
  },
  "keypairs": {
    "kyber": 12,
    "dilithium": 8,
    "sphincs": 2
  },
  "note": "Production mode: Using liboqs (Open Quantum Safe)"
}
```

Policy & Governance Layer

Seraph implements enterprise-grade governance with a **Policy Decision Point (PDP)** that enforces least privilege, action gates, and human-in-the-loop approval for sensitive operations.

Governance Architecture



```
| 6. WHAT approval tier is required? |
|   ↳ auto / suggest / require / two-person |
|_____
```

```
|  ↳ auto / suggest / require / two-person |
```

```
|
|
| POLICY DECISION
|
|
| {
|
|   "permitted": true,
|
|   "approval_tier": "require_approval",
|
|   "rate_limit": 10,           // per hour
|
|   "blast_radius_cap": 5,      // max targets
|
|   "ttl_seconds": 180,        // decision validity
|
|   "allowed_scopes": ["subnet:10.0.1.0/24"]
|
| }
```

```
| {  
|  
|     "permitted": true,  
|  
|     "approval_tier": "require_approval",  
|  
|     "rate_limit": 10,           // per hour  
|  
|     "blast_radius_cap": 5,      // max targets  
|  
|     "ttl_seconds": 180,        // decision validity  
|  
|     "allowed_scopes": ["subnet:10.0.1.0/24"]  
| }  
|
```

```
| "permitted": true,  
| "approval_tier": "require_approval",  
| "rate_limit": 10,           // per hour  
| "blast_radius_cap": 5,      // max targets  
| "ttl_seconds": 180,        // decision validity  
| "allowed_scopes": ["subnet:10.0.1.0/24"]  
| }  
|
```

```
| "approval_tier": "require_approval",  
| "rate_limit": 10,           // per hour  
| "blast_radius_cap": 5,      // max targets  
| "ttl_seconds": 180,        // decision validity  
| "allowed_scopes": ["subnet:10.0.1.0/24"]  
| }  
|
```

```
| "rate_limit": 10,           // per hour  
| "blast_radius_cap": 5,      // max targets  
| "ttl_seconds": 180,        // decision validity  
| "allowed_scopes": ["subnet:10.0.1.0/24"]  
| }  
|
```

```
| "blast_radius_cap": 5,      // max targets
|
| "ttl_seconds": 180,        // decision validity
|
| "allowed_scopes": ["subnet:10.0.1.0/24"]
|
| }
|
```

```
| "ttl_seconds": 180,           // decision validity |
| "allowed_scopes": ["subnet:10.0.1.0/24"] |
| } |
| |
```

```
|     "allowed_scopes": ["subnet:10.0.1.0/24"] |
| } |
| |
```

Approval Tiers

Tier	Description	Example Actions
AUTO	Execute immediately, log for audit	Process list, file hash, network scan
SUGGEST	Show to operator, auto-execute after timeout	Network isolation, file quarantine
REQUIRE_APPROVAL	Must have explicit approval	Process kill, credential reset, patch deploy

Tier	Description	Example Actions
TWO_PERSON	Requires two independent approvals	Mass isolation, key rotation, agent uninstall

Action Categories & Default Permissions

ACTION CATEGORY PERMISSIONS		
OBSERVE • Approval: AUTO • Rate: 1000/hour • Blast radius: ∞ • TTL: 5 minutes	COLLECT • Approval: AUTO • Rate: 100/hour • Blast radius: 50 • TTL: 10 minutes	CONTAIN • Approval: SUGGEST • Rate: 20/hour • Blast radius: 10 • TTL: 5 minutes
REMEDIATE • Approval: REQUIRE • Rate: 10/hour • Blast radius: 5 • TTL: 3 minutes	CREDENTIAL • Approval: TWO_PERSON • Rate: 5/hour • Blast radius: 3 • TTL: 1 minute	DECEPTION • Approval: REQUIRE • Rate: 10/hour • Blast radius: 20 • TTL: 1 hour

Trust-Based Access Control

Trust State	Allowed Actions
trusted	All categories (observe, collect, contain, remediate, credential, deception)

Trust State	Allowed Actions
degraded	Observe, collect, contain
unknown	Observe only
quarantined	No actions permitted

Tool Allowlists by Role

```
TOOL_ALLOWLIST = {
  "agent": [
    "process_list", "process_kill", "network_scan", "file_hash",
    "memory_dump", "network_isolate", "firewall_block"
  ],
  "operator": [
    # Agent tools PLUS:
    "credential_rotate", "agent_deploy", "playbook_execute"
  ],
  "admin": ["*"] # All tools
}
```

High-Risk Action Protection

Certain actions always require elevated approval:

```
HIGH_RISK_ACTIONS = [
  "credential_revoke",      # Revoke authentication tokens
  "mass_isolate",          # Isolate multiple endpoints
  "wipe",                   # Data destruction
  "format",                 # Disk formatting
  "agent_uninstall",        # Remove security agent
  "firewall_disable",       # Disable firewall
  "encryption_key_rotate"   # Rotate master keys
]
```

Audit Trail

Every policy decision is logged with full context:

```
{
  "decision_id": "pdp-a1b2c3d4e5f6",
  "timestamp": "2024-03-12T14:32:00Z",
  "principal": "agent:endpoint-workstation-15",
  "action": "network_isolate",
  "action_category": "contain",
  "targets": ["10.0.1.50"],
  "permitted": true,
  "approval_tier": "suggest",
  "rate_limit_remaining": 18,
  "blast_radius_remaining": 9,
  "decision_hash": "sha256:abc123..."
}
```

Support

- **Repository:** github.com/Byron2306/Metatron
- **Branch:** `mobile` (development), `main` (stable)
- **Issues:** GitHub Issues

Metatron / Seraph AI Defense Platform

Protecting enterprises in the age of autonomous AI threats